

ADTA – 5340: Discovery and Learning with Big Data

Project Group – Big Data 5

**Project Title – Automatic Detection of Fake News Content in
Social Media**

Team Members

Nicholas Cancino - 11667185

Rajkumar Jangeti - 11634942

Sai Anudeep Kashetty - 11631399

Aakash Reddy Varala - 11637262

Ali Shan Lakhani - 11758574

Table of Contents

Abstract	3
Meetings & Communication	4
Introduction	5
Problem Statement	5
Methodology	6
Literature Review	7
Data Description	8
Data Cleaning:	8
Data Preprocessing	8
Exploratory Data Analysis (EDA)	11
Data Visualization	19
Correlation Between Features and Fake News	20
Research Questions	21
Model Building & Model Evaluation	27
Random Forest Classifier	27
Confusion Matrix	28
Navie Bayes Classifier	29
Decision Tree Classifier	31
Logistic Regression	33
K-Nearest Neighbors (KNN) classifier	34
Support Vector Machine (SVM) classifier	36
Challenges Encountered	38
General Evaluation	38
References	41
Appendix	42

Abstract

Information that wasn't previously disseminated in the history of humanity is now possible because of the growth of the Internet and online platforms like Facebook and Instagram. With the increased use of social media platforms, consumers are producing and sharing vast amounts of data and information, many of which are misleading and have no bearing on reality. The automatic categorization of a written article as false or misleading is a difficult problem. People often use e-newspapers, Twitter, Instagram, YouTube, and many more to view the most recent news notifications. Social media fake news has the potential to cause mental illness and doubt in people. Using machine learning techniques, we can determine if the news we've received is true or false.

To identify false news, this study suggests a machine learning method that involves filtering social media data, categorizing the preprocessed data using an algorithm for machine learning, assessing the system that was created, and assessing the outcomes.

We have witnessed how significant deficiencies in the realm of technology have influenced people's lives in recent years of information sharing. The advent of social media has accelerated the spread of false information from antisocial groups throughout the globe. To address the problem, our group intends to utilize the machine learning ideas we have learned in class to solve it and obtain a solution. We first discussed this initiative with a number of our friends and classmates, who helped us to become aware of the problem. It is also evident how confusion has existed in the past and the present, contributing to the loss of life as a result of misinformation spreading via many channels. All things considered, our group hopes to help maintain peace and sanity by using Machine Learning Concepts to distinguish between fake and authentic news.

Meetings & Communication

Meetings were held via Zoom and WhatsApp; Several steps and challenges were discussed through Zoom meetings while WhatsApp was used more to discuss and confirm small details.

Meeting 1 - February 8th, 2024:

We discussed what the project would need, planned steps, recognized team member skills, designated roles, planned meeting dates, and settled on early submission for the project.

Meeting 2 - February 22nd, 2024:

We reviewed datasets, selected primary and backups, analyzed variables, sought TA approval, and scheduled meetings.

Meeting 3: February 26th & 29th, 2024:

The group took some steps to pre-process data, brainstorm visualization, clean data, structure the project timeline, explore Python libraries.

Meeting 4 - 21st March 2024:

Explored Python libraries: NumPy for numerical computations, and Pandas for data manipulation, discussed EDA, visualization, preprocessing, and ML models.

Meeting 5: 02 April 2024:

Explored preprocessing stages, model evaluation, selection, and documentation divisions for the final report.

Meeting 6: 17 April 2024:

Adding the different types of classifiers, analyzing the each model, finalized our report by adding the models summary, and finalized our presentation module, individual assessment of our project.

Introduction

The world is changing swiftly. There are undoubtedly numerous advantages to being a part of the digital era, but there are also disadvantages. In the digital era, new issues surface. One of them is fake news. Someone can easily spread false information. Disinformation aimed toward a specific person, a political party, or an organization might be the source. Fake news may be shared via a number of online platforms. This includes Facebook, Twitter, and so on. Machine learning, a branch of artificial intelligence, is in charge of building computers that can learn and perform a variety of jobs. Machine learning algorithms come in a variety of forms, including supervised, unsupervised, and reinforcement learning algorithms. The first step in training the algorithms is to utilize a set of data called the "train data set". After training, these algorithms may be used to a variety of tasks. Many sectors are using machine learning for a range of activities. Predictions and the identification of hidden objects are two common uses for machine learning algorithms. Online platforms help users since they make it easy for them to obtain news. This, however, raises concerns since it gives hackers access to these sites for the purpose of spreading false information. This news might have negative effects on a person or on society. Readers who have read the news start to trust it without checking further. It is hard to distinguish bogus news, which makes it a major challenge. People will be able to spread the false information to others if it is not found right away, and gradually, everyone will start to believe it. The erroneous information may affect individuals, organizations, or political parties. People's behaviors and views are impacted by fake news, and many studies are working to uncover misleading information. It is evident that machine learning is advantageous in this way. Researchers use a variety of algorithms to spot false information. They have used machine learning to identify fake news. The volume of false information is increasing with time. That's why it's critical to recognize fake news. This objective guides the training of machine learning algorithms. Following training, machine learning algorithms will be able to identify fake news. This literature review will cover the numerous research difficulties. This review of the literature will show how important machine learning is for spotting fake news. We'll also talk about how machine learning is used to identify false news. Machine learning methods used for false news identification will be covered in the literature review.

Problem Statement

The motive of this research is to build a machine learning model that can identify fake news from actual news when it is posted by users on social media platforms. The data for the Truth Seeker and Basic ML datasets came from a crawl of the Politics Dataset's tweets on actual and fake news. Following the acquisition of these ground truth values, a manual procedure including the creation of keywords associated with the news story and their entry into the Twitter API is used to search for tweets on these topics. On social networking sites like Twitter, automatically recognizing fake information is a recurring issue. Theoretically, spotting fake content on social networking sites present a simple binary categorization challenge. The number of tweets shared

every day on social media, to verify their veracity. Using social media applications is a means of becoming popular in today's environment.

Social media platforms such as Instagram, Twitter, and Facebook have evolved into discussion boards, information sharing, and personal communication channels. Users create a massive amount of textual content when they interact with these platforms, such as posts, comments, and private messages. Through this text collection, investigators have the opportunity to discover new information, identify patterns, and compile data that might aid in their understanding of relationships, behaviors, and perhaps criminal activities.

When analyzing text data from social media, there are several issues to consider. The volume of data generated on social media platforms such as Facebook, Instagram, and Twitter first demand the use of data collection, storage, and analysis tools. Second, it may be difficult to interpret written content due to language and cultural idiosyncrasies, as well as the variety brought forth by these platforms' global reach. Finally, it is important to have systems in place for validating and assessing the data sources since concerns about authenticity and truthfulness may raise issues.

First, we use our resources to retrieve the dataset. Pre-processing is the term for the modifications we perform to our data before providing it to the program. Data preprocessing is the process of converting unclean data into a collection of clean data. The dataset must next be processed using our knowledge of Python, and methods for data preparation and visualization must be applied to clean and visually represent the dataset, respectively. Data visualization is an essential ability in machine learning and applied statistics. Quantitative data interpretations and estimations are the primary emphasis of statistics. Since disinformation and fake news are so common on social media sites, we need to figure out how to stop them from spreading. We propose to build a model that can recognize and classify misleading news to counteract it before it reaches consumers on social networks. Furthermore, this model will be universally applicable to a wide range of situations and fields. We will look into the many ways social media users interact with posts on news articles. Understanding the information, producing the visuals, and utilizing a variety of machine learning models can help us resolve this problem by improving our ability to predict if the tweets are real, or they are fake.

Methodology

The technique generally adheres to a set of clearly defined procedures to guarantee a methodical and efficient approach. Data collection is the initial phase, which entails gathering pertinent data from several sources. This procedure is essential since the results of the analysis are directly impacted by the quantity and quality of the data. Following data collection, the next stage is data preparation, during which time the raw data is cleaned up and put into a format that can be analyzed. This covers dealing with outliers, managing missing numbers, and standardizing data types.

After preparing the data, exploratory data analysis (EDA) is carried out to learn more about the properties of the dataset. Utilizing statistical summaries, visualizations, and other exploratory

approaches, this stage looks for patterns, trends, and possible correlations among the five data sets. Decisions about further analysis and model construction are aided by EDA. Model construction is the next stage when different statistical or machine learning models are chosen and trained using the preprocessed data. The objectives of the project and the type of data will determine which models are used.

After the model is constructed, we move onto model assessment as this is the final step where the model is so effective when it is evaluated. It consists of evaluating the model generalization ability for the new and untested data using the criteria that we have set for it. The model evaluation confirms the efficiency of the selected strategies, and it provides insights into the improvements we can make. It is essential to iterate as the process we needed and the improvement of the stages which is based on the analysis and reviews we made. The iterative structure of this method ensures a dynamic approach and adapts to the specifics of the data and the objectives we have placed.

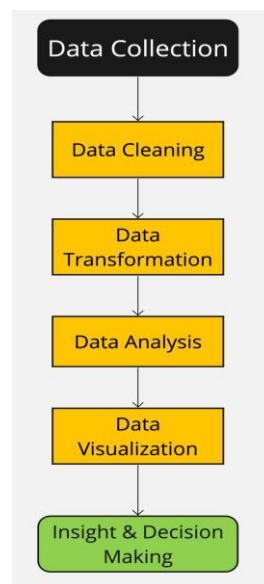


Fig 1 Data Analysis Process

Literature Review

It becomes crucial to look up the methods used in previous studies in the area we wish to focus on before writing the code for the incoming data. Numerous publications that have investigated the identification of false news have been examined by us. Numerous models with various problems and outcomes were trained, and these models were really helpful to us in our endeavor. Numerous techniques, from deep learning algorithms to linear regression algorithms, have been used by the researchers. The first point made in every article was the long-standing problem of false news, which has caused much disruption and, in many cases, even fatalities. They have discussed the significance of classifying such news and the necessity of eliminating such propaganda to avoid misrepresenting false material as news. The process of transforming data into a valid dataset and demonstrating the application of NLP has been covered in the context of data cleaning. Various

types of algorithms, including SVM and Random Forest, have been covered in research articles. The use of deep learning algorithms, such as CNN, has been demonstrated, and the ultimate accuracy has been demonstrated when significance to data categorization has been assigned.

Data Description

Approximately 130,000 tweets make up this dataset, which is divided into 700 subjects that are real and 700 fake topics. Features such as the number of followers, favorites, responses, mentions, and friends were gathered from Twitter and the user's metadata when they tweeted the material. The majority of individuals who agree with the tweets are classified by Amazon Mechanical Truck, which determines the tweet's truth value. Because of the quantity of input characteristics, conventional machine learning models may be constructed using this dataset.

Nevertheless, there are several drawbacks to this dataset, such as its dependence on Amazon Mechanical Truck for data labeling and the possibility of bias or human mistakes in certain circumstances. The whole dataset solely includes information from Twitter platform tweets.

Data was taken from *Truth Seeker Dataset 2023 | Datasets | Research | Canadian Institute for Cybersecurity | UNB*. (n.d.). <https://www.unb.ca/cic/datasets/truthseeker-2023.html>

Data Cleaning:

The dataset has undergone several modifications on our part to make the model easier to train and produce reliable results. The data has been combined into a single kind. Furthermore, by importing the panda's package we were able to enhance the data by eliminating the features with no predictive relationships for fake news tweets. For instance, the following columns have been removed: index, statement id, state information, context, rationale, numbers of barely true, false, half true, largely true, and pants on fire. After that, we binary classified our data, treating true, half-true, and mainly true as real (1) and false, pants-false, and barely true as fake (0). For improved analysis, we then combined the statement and subject column into one. We removed any missing values and then changed them to lowercase if they were present. Approximately 130,000 tweets make up this dataset, which is divided into 700 subjects that are real and 700 fake topics. Features such as the number of followers, favorites, responses, mentions, and friends were gathered from Twitter and the user's metadata when they tweeted the material.

Data Preprocessing

One of the most important steps in the data analysis pipeline is the data preparation process, which is preparing raw data for analysis or machine learning by cleaning and formatting it. In this case, we are dealing with outliers, duplicate data, and missing values. Managing null in a dataset, values serve as placeholders to indicate whether a certain variable in an observation has a value or not. They are also known as NaN (Not a Number) values or missing values.

Handling null values is a crucial component of preprocessing and data cleaning. Our project's procedures to handle null values are shown below.

We must vectorize and transform our textual data into numerical data because it is currently in that format. We will use natural language processing to assist with this. Therefore, to truly transform the language into numbers, we must first clean up the data. Using standard procedures, we first eliminate any links, unnecessary white space, and punctuation (apart from commas). We tokenize the data in our initial NLP, which comes next. We utilize the library for tokenization since it breaks down phrases and paragraphs into smaller parts and gives them meaning. The text has since been lemmatized, which essentially implies that any word may be changed to its root node, therefore grouping the text has since been lemmatized, which essentially implies that all words should be changed to their root node to group words with similar meanings. Reducing the suffices is how this is accomplished.

The sentence has also been freed of stop words like "and" and "the," which don't contribute anything significant. After that, we applied NLP techniques to turn the text into a numerical number by combining the text column into a string for vectorization. In order to view the data, we have also represented the frequencies with a word cloud. Our next job is to use the NLP algorithms bag of words (BOW) and term frequency-inverse document frequency (TF-IDF). These techniques basically help vectored the data, which is the process of turning textual data into numerical data. Each dictionary key in BOW is associated with a word, and the value denotes the word's frequency of occurrence. On the other hand, TF-IDF quantifies the significance and pertinence of a string representation (words, phrases, and lemma) in a document within a collection of documents. Next, we use a label encoder to convert the label to a numerical format so that the machine can read it.

```
In [8]: # Displaying null values count for each column
for column, null_count in null_counts.items():
    print(f"{column}: {null_count}")
```

```
Unnamed: 0: 0
majority_target: 0
statement: 0
BinaryNumTarget: 0
tweet: 0
followers_count: 0
friends_count: 0
favourites_count: 0
statuses_count: 0
listed_count: 0
following: 0
embeddings: 0
BotScore: 0
BotScoreBinary: 0
cred: 0
normalize_influence: 0
mentions: 0
quotes: 0
replies: 0
retweets: 0
favourites: 0
hashtags: 0
URLs: 0
unique_count: 0
total_count: 0
ORG_percentage: 0
NORP_percentage: 0
GPE_percentage: 0
PERSON_percentage: 0
MONEY_percentage: 0
DATE_percentage: 0
CARDINAL_percentage: 0
PERCENT_percentage: 0
ORDINAL_percentage: 0
FAC_percentage: 0
LAW_percentage: 0
PRODUCT_percentage: 0
EVENT_percentage: 0
TIME_percentage: 0
LOC_percentage: 0
WORK_OF_ART_percentage: 0
QUANTITY_percentage: 0
LANGUAGE_percentage: 0
Word count: 0
Max word length: 0
Min word length: 0
Average word length: 0
present_verbs: 0
past_verbs: 0
adjectives: 0
adverbs: 0
adpositions: 0
pronouns: 0
TOs: 0
determiners: 0
conjunctions: 0
dots: 0
exclamation: 0
questions: 0
ampersand: 0
capitals: 0
digits: 0
long_word_freq: 0
short_word_freq: 0
```

Fig 2: Null Values.

In data cleaning and preprocessing, handling duplicate values in a dataset is crucial. Multiple values can occur for several reasons, including mistakes in data entry, malfunctions in the system, or the merging of information.

Exploratory Data Analysis (EDA)

The essential phase in the data analysis process is the exploratory data analysis. To detect the data qualities, highlight the trends, and get the main insights it is very important to analyze and display the data correctly. Data points that differ from the observations in the dataset which we call outliers. The statistical analysis we do is regarding the high numbers which are affected by the machine learning models' accuracy. Measurement mistakes or data variability can lead to outliers. One tactic is to lessen the influence of outliers by swapping them with the median.

The median denotes the center value in the dataset and will be less impacted by the outliers which will be different from the mean, which is sensitive to high values. By replacing the outliers with the median, we can prevent extreme results. Where there are skewed distributions the dataset where the outliers are present might be the false findings. This method is quite helpful in a variety of domains such as statistics, and machine learning. The substitution of median in place of outlier results is making the data more stable and more trust in the dataset and its analysis.

```
In [12]: # Selecting numerical columns from the DataFrame
numerical_columns = data.select_dtypes(include=['int64', 'float64'])
# Printing the numerical columns
numerical_columns
```

Out[12]:

Unnamed: 0	BinaryNumTarget	followers_count	friends_count	favourites_count	statuses_count	listed_count	following	BotScore	BotScoreBinary
0	0	1.0	4262.0	3619.0	34945.0	16423.0	44.0	0.0	0.03
1	1	1.0	1393.0	1621.0	31436.0	37184.0	64.0	0.0	0.03
2	2	1.0	9.0	84.0	219.0	1184.0	0.0	0.0	0.03
3	3	1.0	4262.0	3619.0	34945.0	16423.0	44.0	0.0	0.03
4	4	1.0	70.0	166.0	15282.0	2194.0	0.0	0.0	0.03
...	...	...	...	...	...	...	...	...	...
134193	134193	0.0	2075.0	2662.0	156306.0	62375.0	3.0	0.0	0.03
134194	134194	0.0	1636.0	1626.0	215290.0	44011.0	10.0	0.0	0.03
134195	134195	0.0	266.0	2501.0	28377.0	7617.0	1.0	0.0	0.03
134196	134196	0.0	2.0	45.0	16.0	509.0	0.0	0.0	0.03
134197	134197	0.0	178.0	471.0	1254.0	2094.0	0.0	0.0	0.03

134198 rows × 60 columns

Fig 3

Fig 3 shows us the 134198 rows and 60 columns in the dataset. There are 3 columns as the string data type column and the rest are the numerical columns. The column statements have 1058 unique values, and the column tweet has the largest unique values of 134198 unique values. The last column embedding has 105 unique values. The counts have the datatype of an integer in every case.

An essential part of EDA is comprehending the data distributions. The dataset distributions characterize the manner and the form in which the values are being distributed. Different

distribution types have unique forms, and an understanding of the distributions might provide details about the data composition.

Fig 4

Fig 4 highlights the models and the count of each type in the dataset.



The process of removing the outliers highlighting them and checking whether we have an accurate trustworthy dataset.

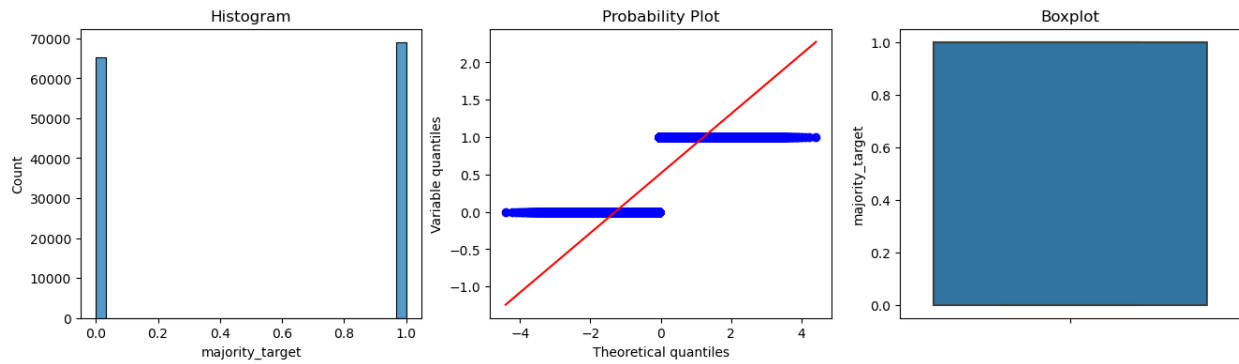


Fig 5

Fig 5 shows us the histogram analysis as well as the probability plot for the majority target in the dataset.

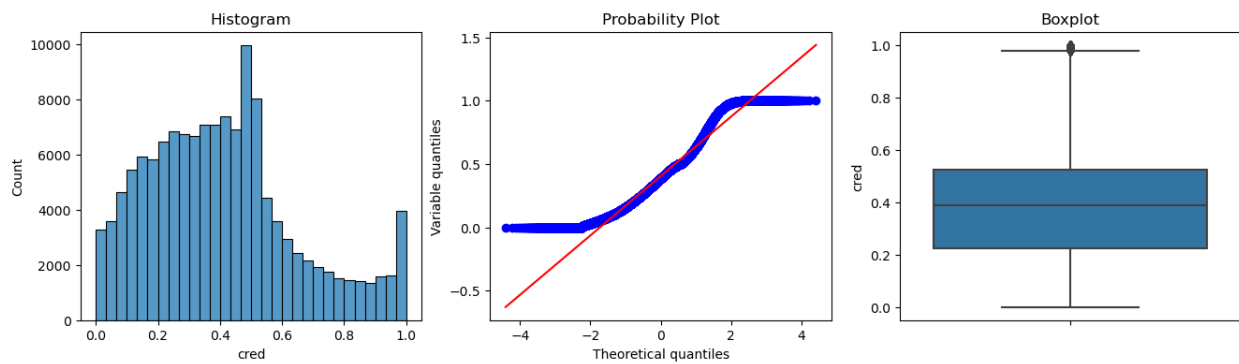


Fig 6

Fig 6 shows us the histogram analysis for the cred data in the dataset.

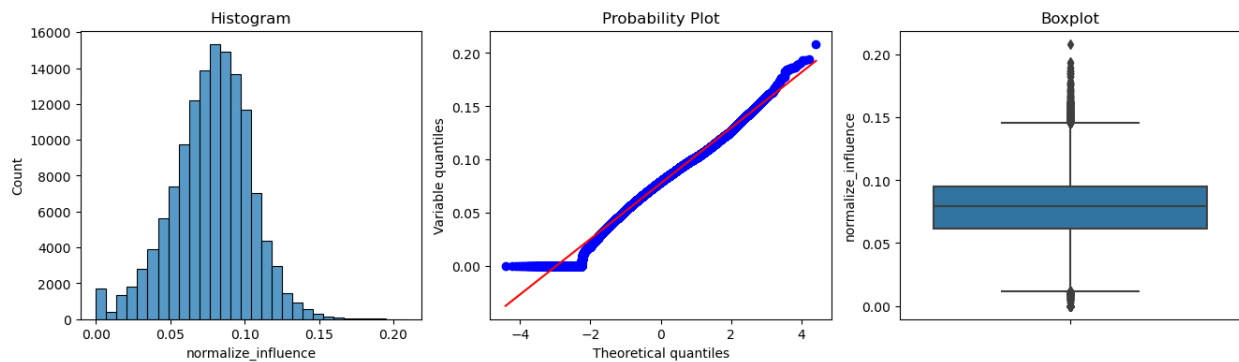


Fig 7

Fig 7 shows us the histogram analysis as well as the boxplot analysis of the normalize influence through the dataset.

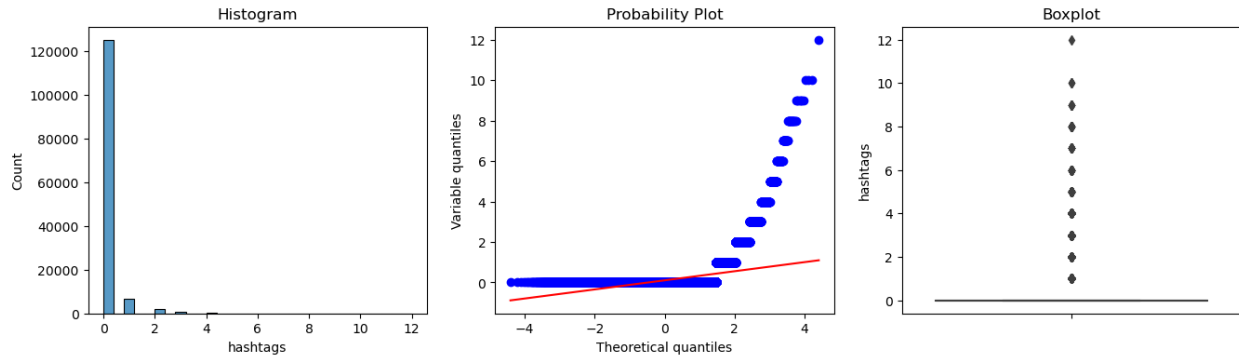


Fig 8

Fig 8 shows us the analysis part for the hashtags used in the dataset.

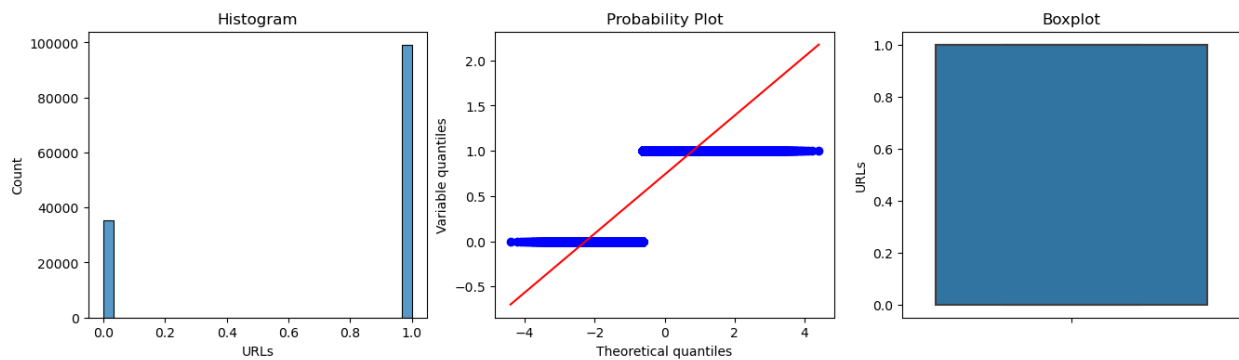


Fig 9

Fig 9 shows us the analysis of the URLs used in the datasets through the histogram and boxplot analysis.

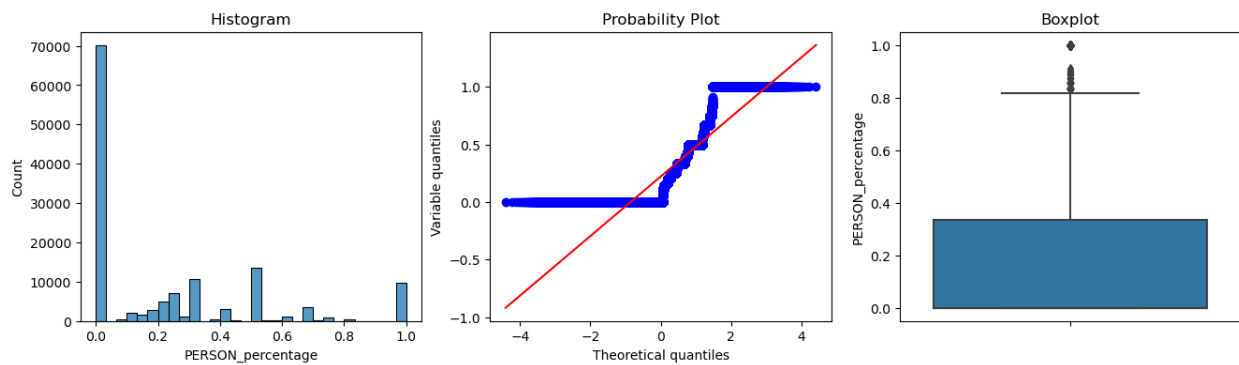


Fig 10

Fig 10 highlights the person percentages that are involved in the dataset.

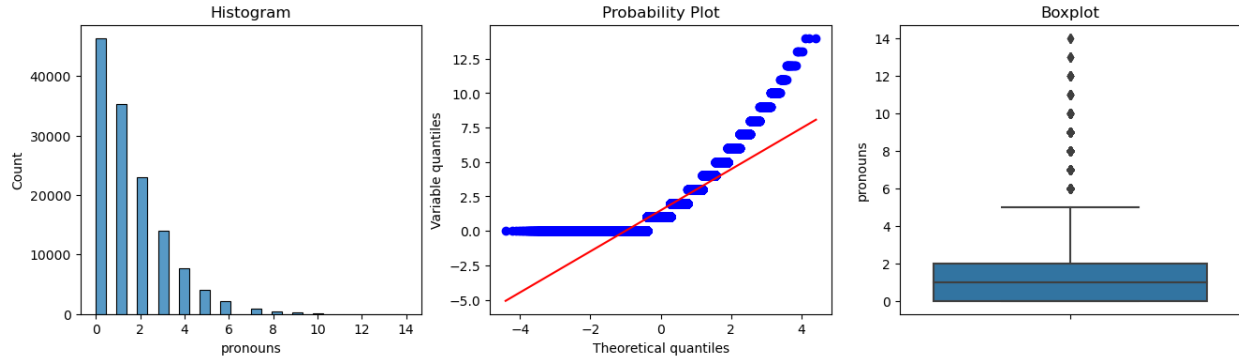


Fig 11

Fig 11 highlights the pronoun data used in this dataset through the statistical analysis approach.

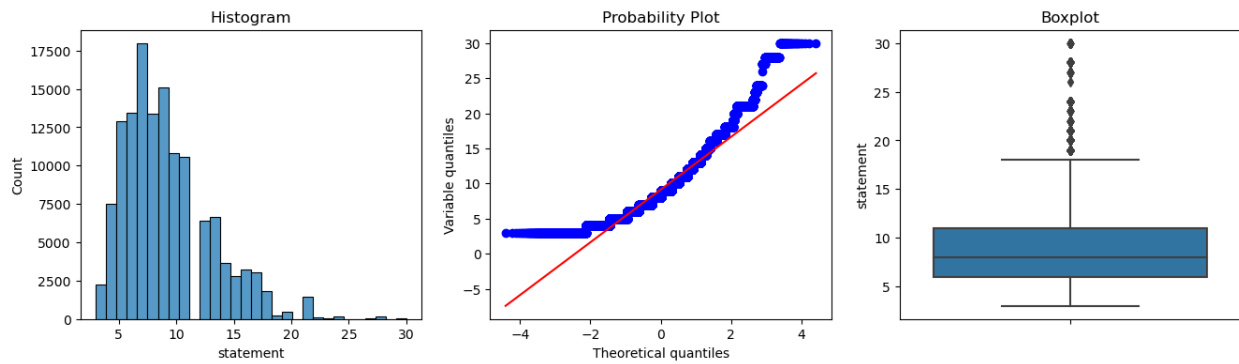


Fig 12

Fig 12 shows us the statement functions used in the dataset using the histogram and other statistical tools.

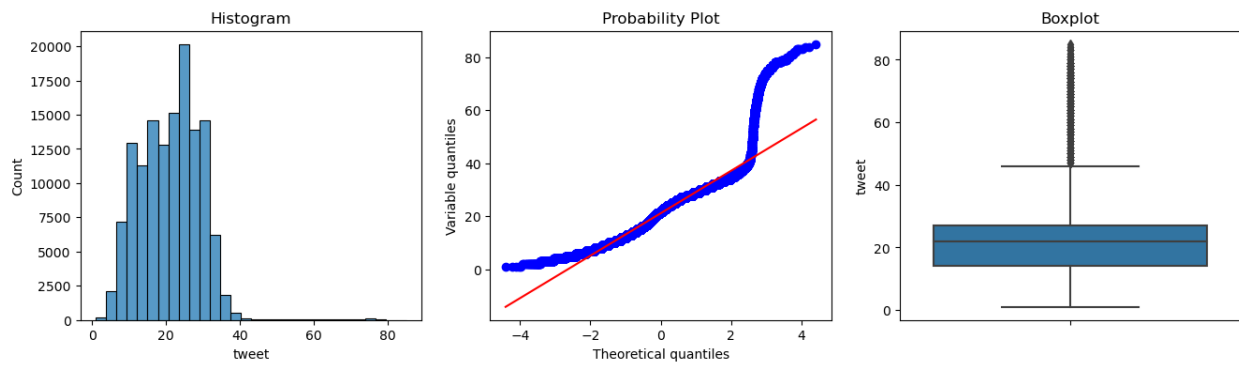


Fig 13

Fig 13 shows us the statistical analysis of the tweet features in the dataset.

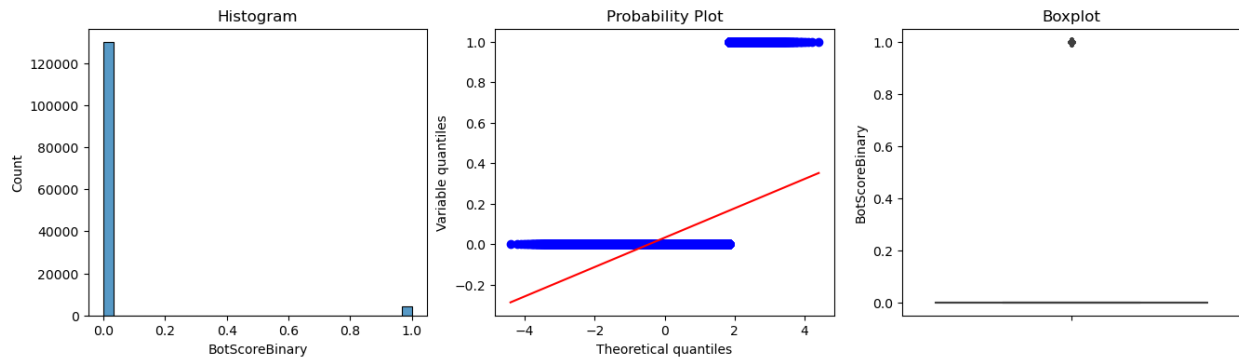


Fig 14

Fig 14 Shows us the bot score binary analysis from the dataset.

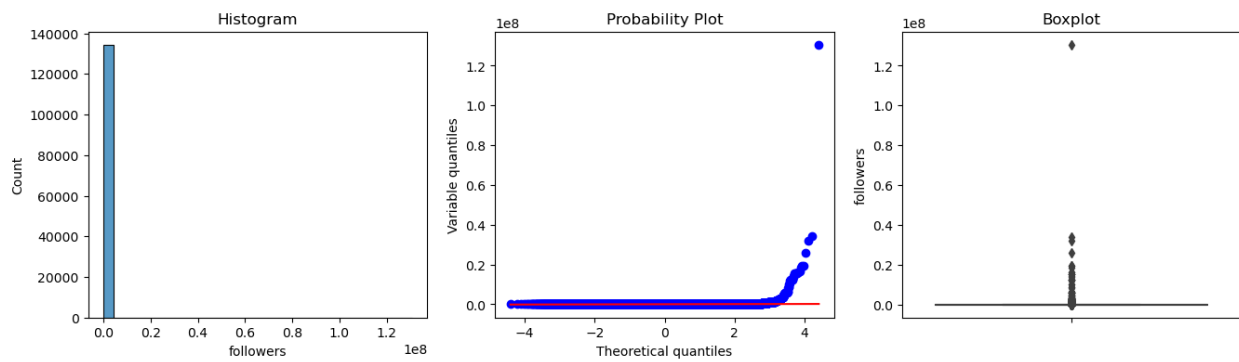


Fig 15

Fig 15 highlights the followers that are presented in the dataset for the analysis features.

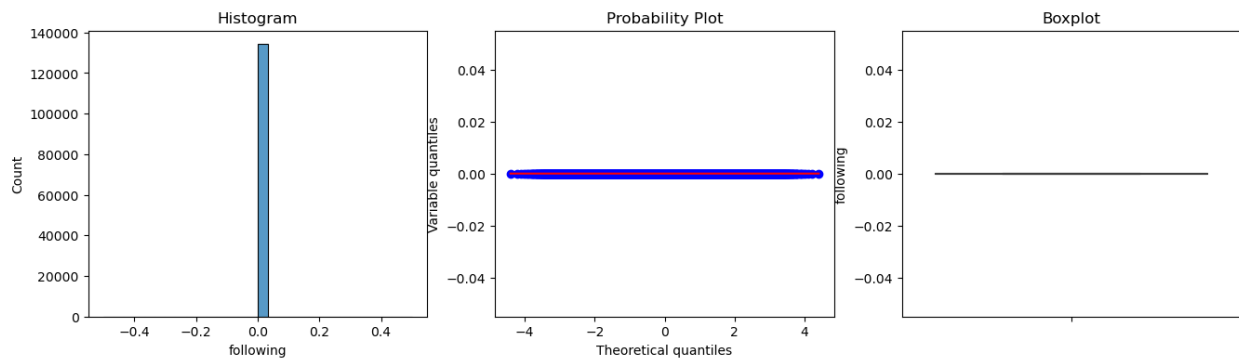


Fig 16

Fig 16 shows us the following included in the dataset with no outliers found here.

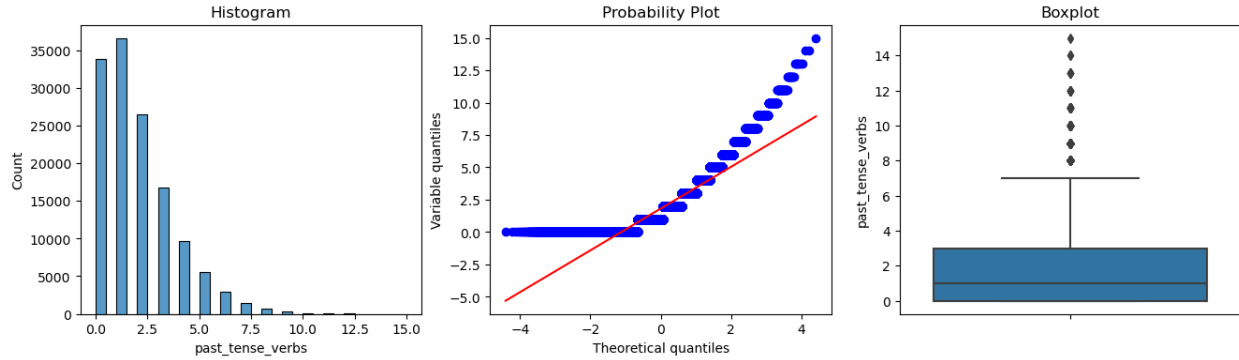


Fig 17

Fig 17 highlights the past tense verbs used in the dataset and as gradually it decreases in the count as more past tense verbs are used.

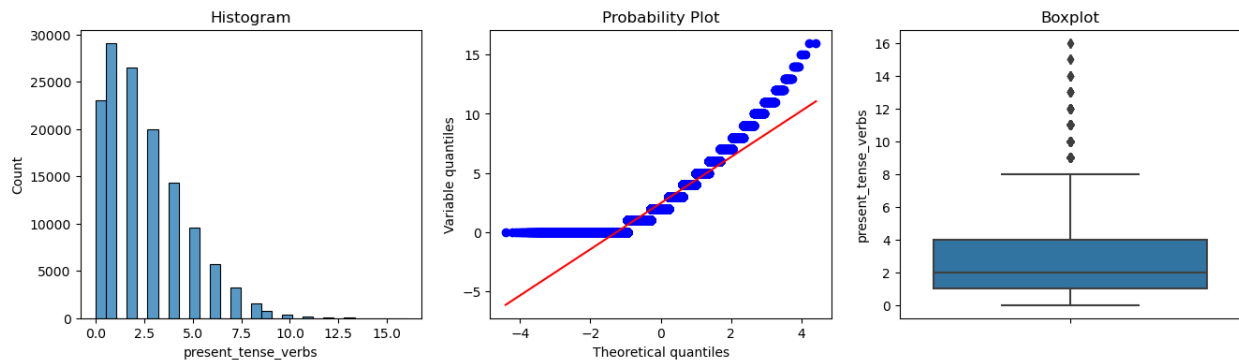


Fig 18

Fig 18 illustrates the present tense nature of our dataset. Through it, we can make words used in the present tense go as the inversely proportion phase.

Through the exploratory data analysis, we can come to the conclusion that there are few outliers in the variables mentioned above.

Models:

We used models to get the highest accuracy and ended up with the random forest classifier as the best model. Graphs such as the confusion Matrix show the actual value compared to the predicted value demonstrating the relative accuracy of models.

Analysis:

For each feature, we used graphs like box plots so we could get an idea of what sort of data we could use and which ones we could drop compressing the data set and selecting the best features having a less complex.

The analysis of trimming the data now begins with trimming and identifying the outliers.

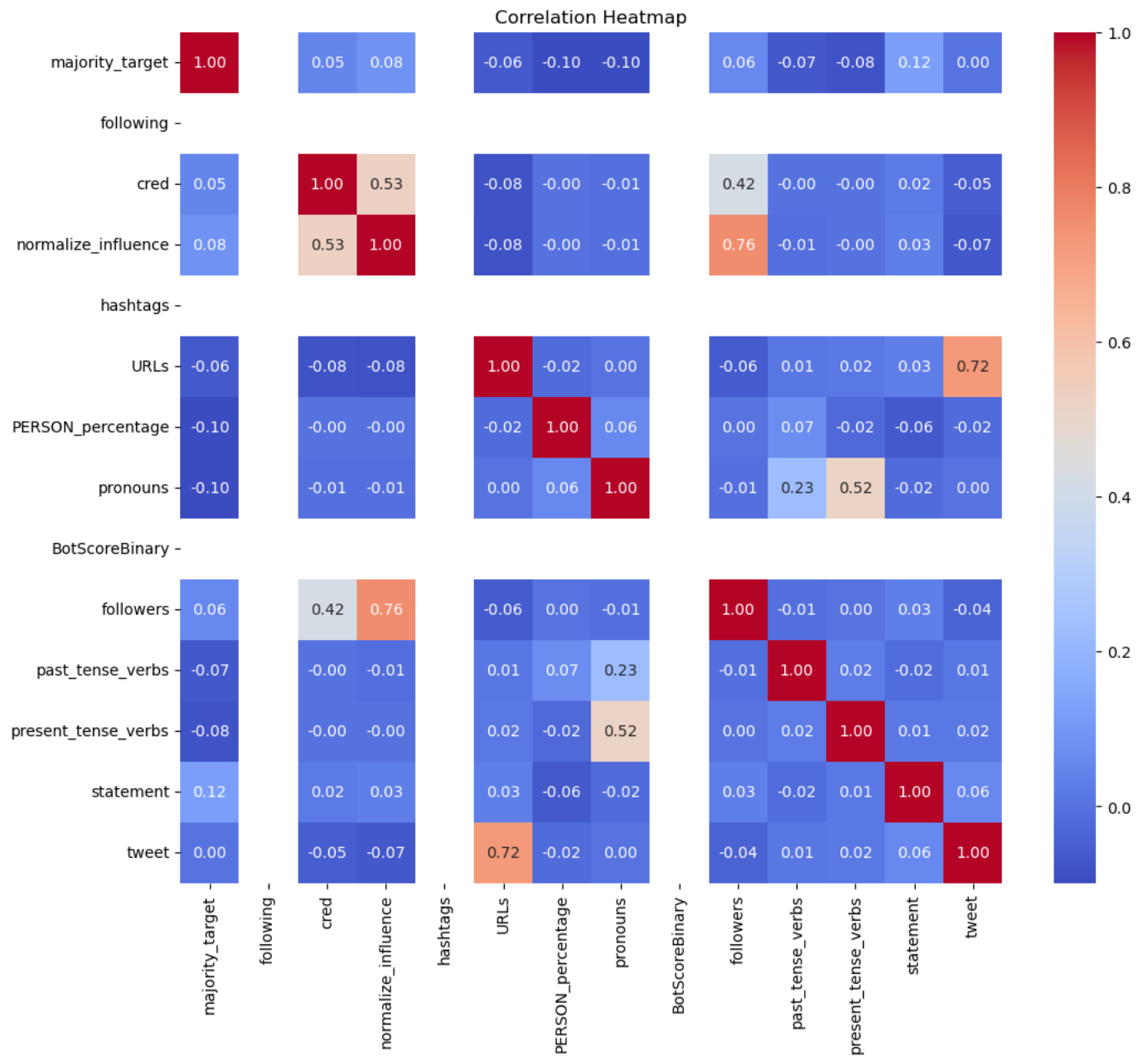


Fig 19

Fig 19 shows us the correlation of the excluding and the trimming of the non-numeric columns and calculating the correlation matrix with the response of the variables used in the dataset.

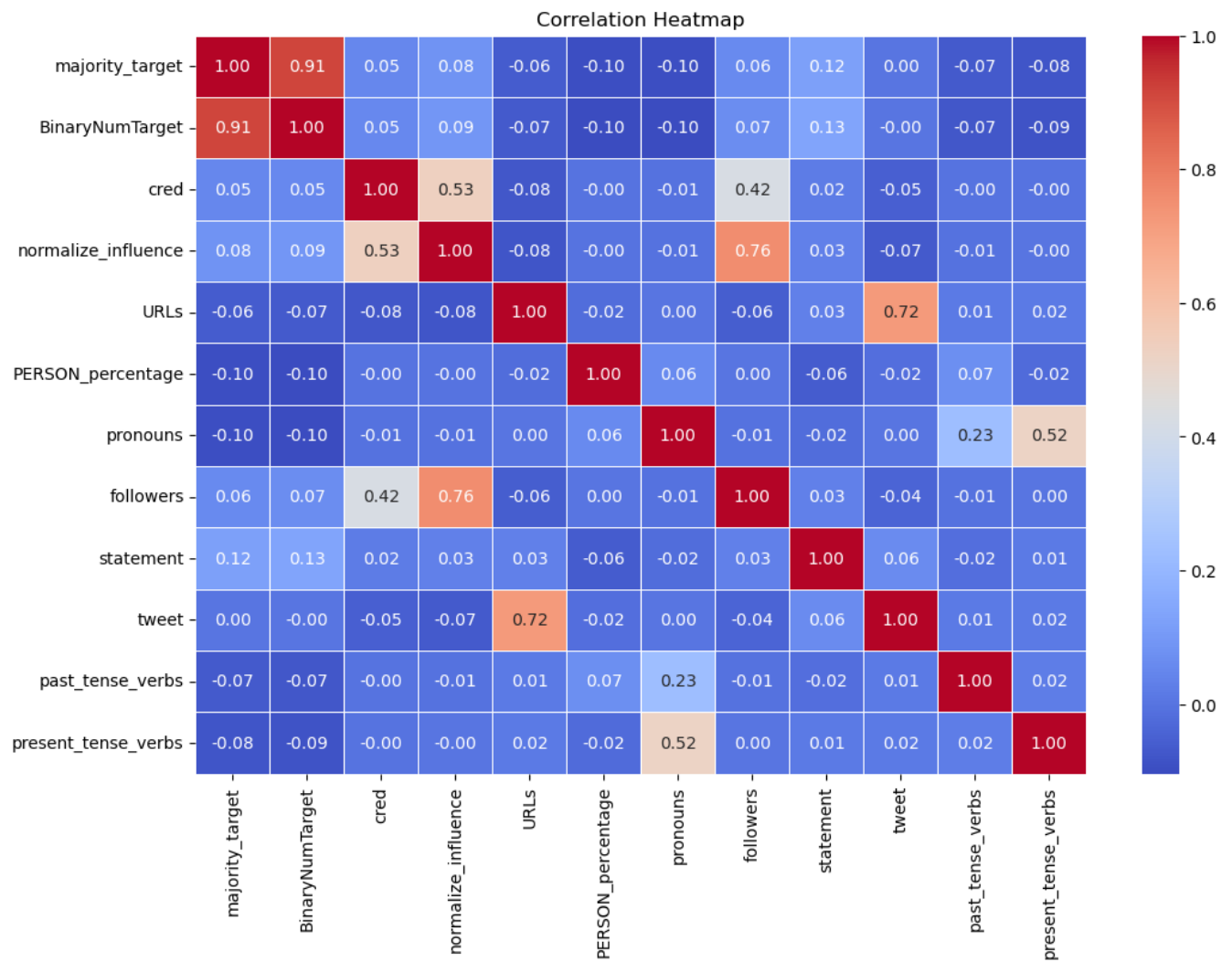


Fig 20

Fig 20 highlights the designed data columns 'majority_target', 'BinaryNumTarget', 'cred', 'normalize_influence', 'URLs', 'PERSON_percentage', 'pronouns', 'followers', 'statement', 'tweet', 'past_tense_verbs', 'present_tense_verbs with the response of the correlation heat map.

Data Visualization

Because data visualization can convert complicated facts into comprehensible and informative graphical representations, it is extremely important for data analytics initiatives. Analysts and stakeholders may quickly obtain significant insights thanks to data visualization, which shows data patterns, trends, and correlations visually. Visualizations make difficult material easier for non-technical audiences to understand by communicating results understandably and intuitively.

The data visualization makes the trends easier to visualize, and to show the outliers and the possible linkages that are not obvious in the raw data. It is an effective way to develop and validate the hypotheses and do more research. A better visualization of the situation can be compelling and can be easily presented through visuals, which improves the decision-making process and the information to form the decisions. In the context of data analytics, the initiatives are essential to be taken into consideration when making the visualizations for having actionable insights, improving communication, and having the correct decision-making.

Correlation between Features and Fake News

The research team examined the data set of 63 features, each of which was created to capture distinct facets of tweets that might be associated with fake news on social media sites. Engaging in metrics, like the number of followers, recorded account of credibility, and titles of tweets were investigated because they might reveal whether or not a tweet classifies as a fake news tweet. Furthermore, specific characters in tweets such as quotes, hashtags, and several other features were analyzed to look for correlation in specific character usage to potentially distinguish between real or fake news tweets.

The research also delved into accounts metadata such as number of followers, tweet headlines, and the credibility of the user that posted a tweet. Developing efficient detection techniques requires an understanding of the traits and actions of the users who spread false information. Furthermore, vocabulary variety and sentiment analysis were used to look for patterns in language usage that could help distinguish between real and fake news, for instance past tense verbs and present tense verbs are analyzed.

By considering a broad array of features and employing advanced analytical techniques, the research team aimed to construct a robust predictive model capable of accurately identifying fake news posts on social media. Such a model holds promise for bolstering efforts to curb the spread of misinformation and safeguard the integrity of online information ecosystems.

Research Questions

Question 1: How do the credibility scores of users correlate with the prevalence of fake news tweets? Do users with higher credibility scores tend to share fewer fake news tweets?

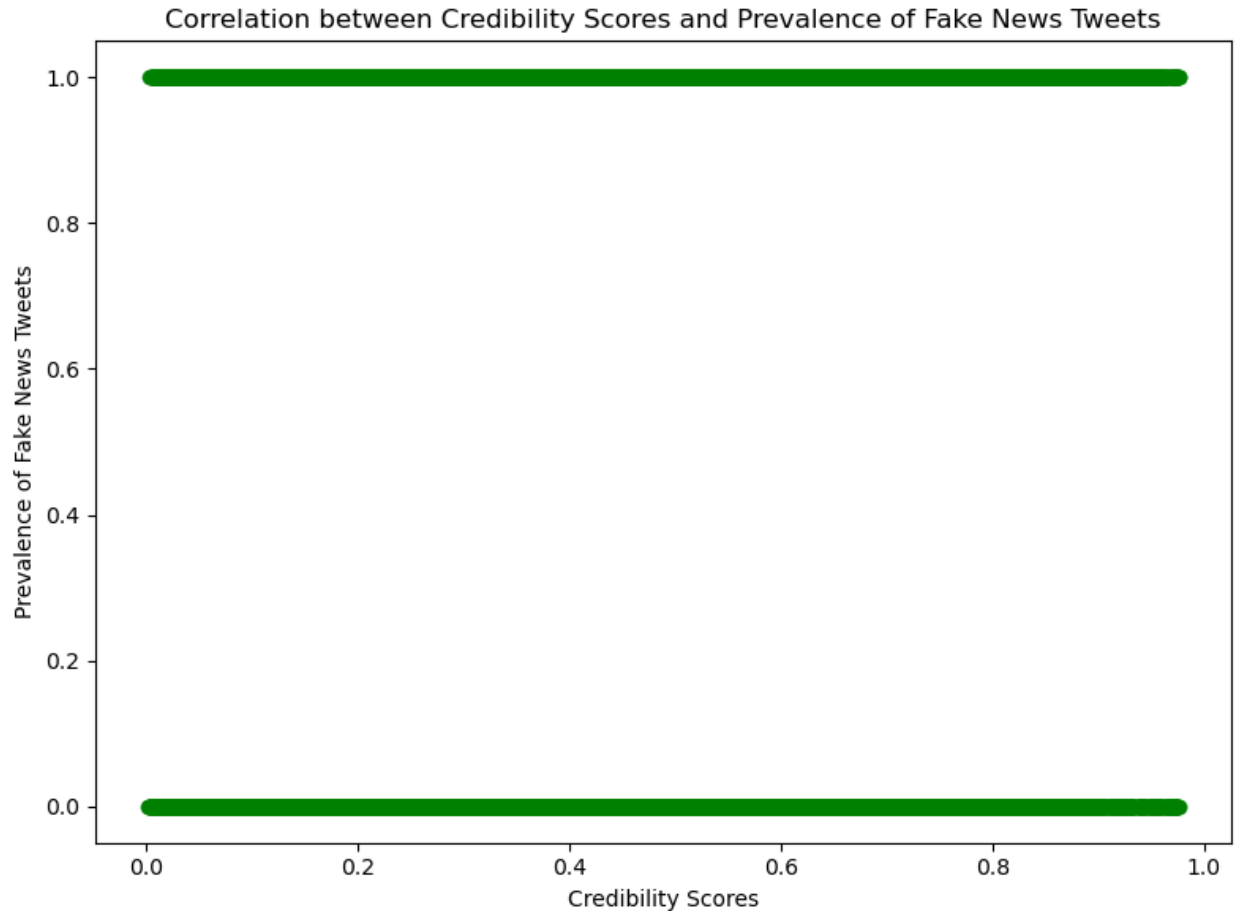


Fig 21

In the above Fig 21 scatter plot there is no connection between the number of false news tweets and credibility scores. The y-axis displays the frequency of false news tweets, while the x-axis displays the credibility score. We initially made predictions on which features would have a strong relationship with fake news. One of the first predicted pivotal features was the credibility score; where the lower the credibility score the more likely hood that the tweet would be false information. On the other hand, accounts with high credibility would be less likely to spread fake news, like an established news outlet tweets like CNN. However, after preprocessing the data set and conducting exploratory data analysis (EDA), the research highlighted the zero connection between the credibility scores of accounts and the presence of fake news tweets. Therefore, we rejected our initial hypothesis and concluded that credibility was not a pivotal feature due to the zero correlation. This outcome was unexpected from the data which underscored the importance of thoroughly analyzing and confirming every expectation of a features relationship between predicting fake news tweets.

Question 2: Are there any particular combinations of features that demonstrate strong predictive power for identifying fake news tweets?

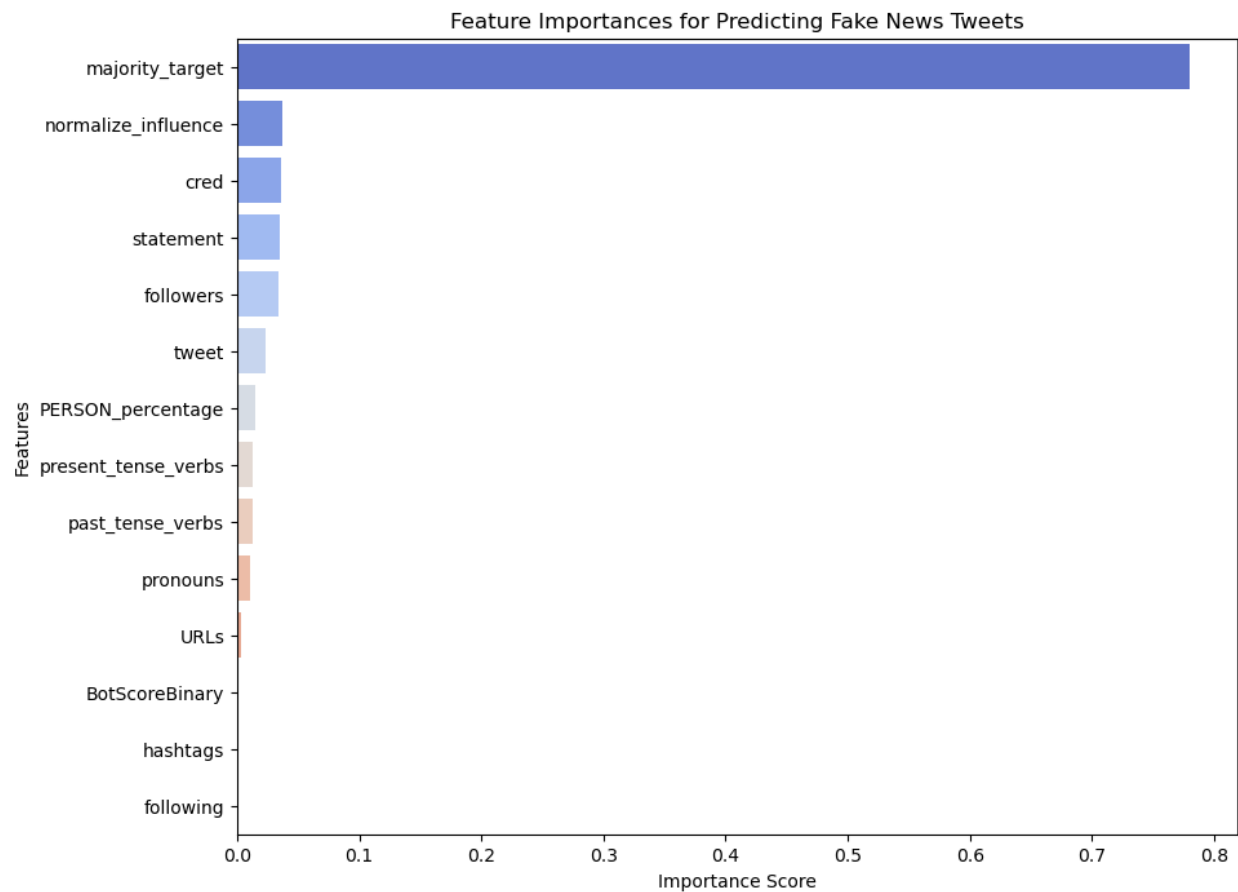


Fig 22

The Fig 22 graph shows that majority_target is the most significant feature, followed by normalize_influence, cred, followers, and statement. Such characteristics are probably technical features used to teach a machine learning model to recognize tweets that are fake news.

Question 3: What is the probability that a fake news tweet is a bot and the correlation between them?

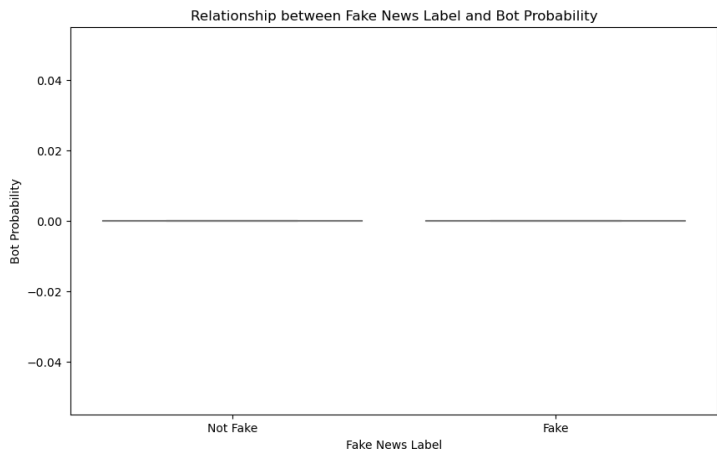


Fig 23

The research team had a keen interest in investigating the role of bots in the spread of fake news on social media platforms. Through their analysis, they sought to determine if there was a correlation between bot activity and the authenticity of news content in social media posts. Surprisingly, the research revealed that fake news tweets had a 0.0 likelihood of originating from a bot. This indicates that, based on the available data, no instances of fake news tweets were detected as being generated by bots. This unexpected discovery counters our initial assumption about the relationship between bot accounts and their usage in tweeting fake news on social media. Moreover, the research into the bots provided notable insight into the underlying details of misinformation on social media platforms. By finding the distinct behavior of bots' presence in tweets, research shows that we can no longer consider the tweet fake news, researchers could alter their approach and analyze and develop more prediction models that take into account bots to detect fake news tweets. With this discovery, our understanding of the complex features that can classify a fake news tweet has advanced notably.

Question 4: What are the main features within the dataset that will be utilized to classify the tweets?

BinaryNumTarget	1.000000
majority_target	0.914422
statement	0.134804
PERSON_percentage	0.104241
pronouns	0.102398
normalize_influence	0.091342
present_tense_verbs	0.085088
past_tense_verbs	0.074887
URLs	0.067455
followers	0.066281

```
cred          0.054722
tweet         0.000655
following      NaN
hashtags      NaN
BotScoreBinary NaN
Name: BinaryNumTarget, dtype: float64
```

The correlation coefficients represent the degree and orientation of the linear connection between every characteristic and the target variable 'BinaryNumTarget', which is probably a measure of whether or not a tweet is considered to be fake news.

When the correlation value is 1.0, it indicates an ideal positive correlation, meaning that the target variable grows proportionately to the growth in the feature value.

A correlation value of -1.0 denotes an ideal negative correlation, meaning that the target variable falls proportionately as the feature value rises.

There is no linear association between the characteristic and the target variable when the correlation value is 0.

Examining the correlation coefficients:

With 'BinaryNumTarget','majority_target' has the strongest positive correlation (0.914422), signifying a robust positive association. This implies that the designation of a tweet as false news is closely related to the target's majority vote.

Though they are less strong than those of "majority_target," "statement," "PERSON_percentage," "pronouns," "normalize_influence," "present_tense_verbs," and "past_tense_verbs" also exhibit positive correlations with "BinaryNumTarget.". These features serve as technical benchmarks to be utilized in training machine learning models to effectively distinguish between genuine and false news content on social media platforms.

'BinaryNumTarget' and features like 'URLs', 'followers', and 'cred' exhibit less positive associations.

The features 'following', 'hashtags', and 'BotScoreBinary' that have NaN correlation values show no linear connection to 'BinaryNumTarget' in the present analysis.

Question 5: How does the relationship between several followers and following contribute to fake news tweets?

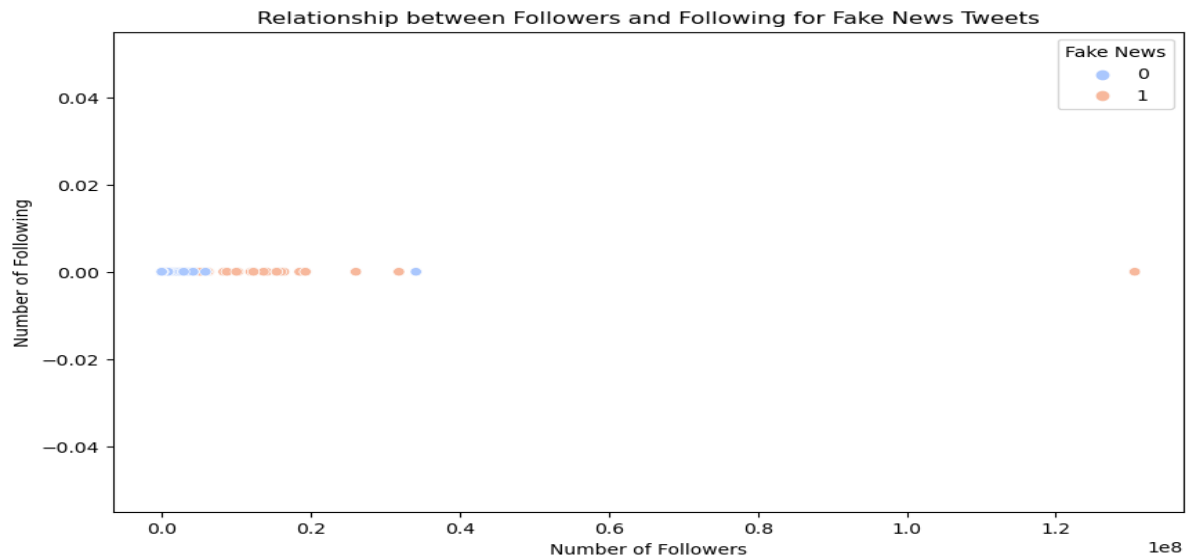


Fig 24

According to Fig 24, there is little negative link between the number of followers and the number of following for accounts that disseminate false information on Twitter. Accordingly, fake news-spreading accounts typically have a smaller following than regular accounts. This revelation casts doubt on presumptions regarding the prevalence and effects of fake news on social media. Additionally, since accounts with smaller followings might have less influence, it raises the possibility of potential limitations in the spread of false information. In an effort to counteract misinformation online, recognizing this trend can help develop more focused strategies for spotting and stopping the spread of fake news on Twitter and other social media platforms.

Question 6: Are tweets with a higher percentage of past tense verbs more likely to be classified as fake news compared to tweets with a higher percentage of present tense verbs?

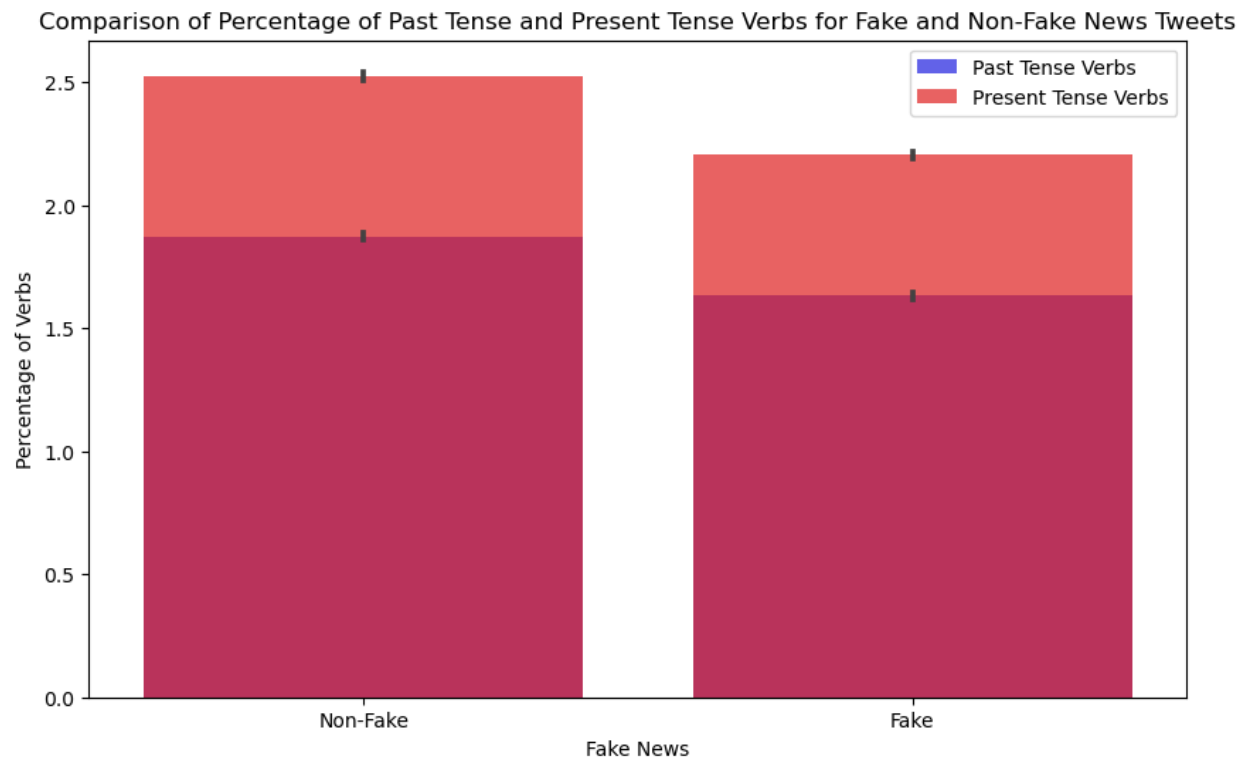


Fig 25

The Fig 25 graph shows that tweets with a larger percentage of verbs in the past tense are more likely to be labeled as false news than tweets with a higher percentage of verbs in the present tense.

The comparison graph shows the proportion of verbs in the past tense and present tense in tweets containing false news and those without. The "Percentage of Verbs" label on the y-axis indicates the proportion of verbs together. Both the "Non-Fake" and "Fake News" categories are shown on the x-axis for the tweets. The proportion of verbs in the past tense is greater in tweets containing false news than in tweets containing real news. Yet the queue for tweets containing false information is constantly greater than the queue for tweets containing actual data. As a result, compared to tweets that aren't false news, the percentage of past tense verbs in fake news tweets is higher.

In the data analytics project, the correlation is very important as it sheds light on the connections between the various variables we have shown in the data set. Determining the strength and the direction of the correlation between the variables is important to form the choices and come up with conclusions. Analysts can determine the patterns or possible relationships by the intensity and the correlation it possesses. This information is really crucial for the feature selection when developing a model. The correlation analysis will help identify multicollinearity, which is a condition in the regression model. Independent variables are having a high degree of correlation

which can affect the model-independent variable with a high degree of correlation. This will also affect the model's interpretability and its ability.

In the end, gaining a thorough grasp of correlation is essential to deriving significant conclusions, improving the readability of statistical models, and supporting data-driven decision-making across a range of data analytics applications.

Model Building & Model Evaluation

In data analytics, model building links with the process of creating the mathematical demonstration of a system from a dataset. In this process, a predictive model is which can identify the patterns and relationships within the data and has to train in. The main focus is to create a model that can accurately classify the data and generate new data. Model creation involves several activities. This is an important stage in the model of data analytics for the workflow when the chosen model is adjusted and verified to make sure it is working. A data analytics project's ability to succeed is the creation of models that can capture the pattern and the basic structure that can be found in data.

In the data analytics project, the model assessment is very important as it is a crucial way that how reliable the model structure can be. Testing can guarantee that the model chosen can accurately show fresh, untested data and boost trust. Analysts can comprehend the model's strengths and possible shortcomings by using the assessment measures of the iterations in the model performance. The assessment stage directs the model-building process by helping to identify areas for development and refinement. Robust model assessment is essential for choosing the model and the range of algorithms correctly, which can maximize the project's overall efficiency. It also provides the decision-making context given by stakeholders creating a clear sense of the model's dependability and fulfilling the final model offers and the insights linked with it based on the underlying data we can align with our project goals.

Random Forest Classifier

Machine learning uses the random forest learning approach for both regression and classification applications. It functions by building numerous decision trees during training and produces the average prediction or classification during the individual tree's regression phase for increased performance and accuracy. The main idea behind the random variables is introduced at two levels: first, bootstrapped samples from the dataset are used to enhance the overfitting of the model, and subsequently, the random subset feature of each tree is chosen. When creating a random forest, the method of combining the predictions from several trees is frequently more reliable and produces an accurate model, making it a good option for a variety of applications across domains.

Random Forest Classifier

```
In [117]: # Initialize Random Forest classifier
rf_classifier = RandomForestClassifier(random_state=42)

# Fit the classifier to the training data
rf_classifier.fit(X_train, y_train)
```

```
Out[117]: RandomForestClassifier(random_state=42)
In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.
```

Fig 26

Fig 26 shows us the random forest classification process.

Confusion Matrix

```
In [118]: # Predicting the target variable on the test set
y_pred = rf_classifier.predict(X_test)
```

```
In [119]: # Calculating accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.9565817058369194
```

```
In [120]: # Printing confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(conf_matrix)
```

```
Confusion Matrix:
[[8735  389]
 [ 398 8604]]
```

Fig 27

Fig 27 shows us the confusion matrix with the percentage going up to 95.65%.

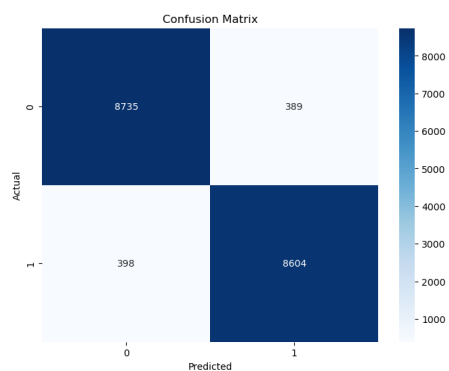


Fig 28

Fig 28 Shows us the confusion matrix demonstration.

```
In [122]: # Printing classification report
class_report = classification_report(y_test, y_pred)
print("Classification Report:")
print(class_report)
```

```
Classification Report:
      precision    recall  f1-score   support

     0       0.96      0.96      0.96     9124
     1       0.96      0.96      0.96     9002

 accuracy      0.96
 macro avg     0.96
 weighted avg  0.96
```

Fig 29

Fig 29 shows us the classification report of the confusion matrix with precision and percentage up to 96%.

Navie Bayes Classifier

This post may include false information about a person or business. It's not easy to find this type of false news, but it's important. The researchers used all three machine-learning algorithms to finish this project. The SVM, neural network, and Naïve Bayes are these. The accuracy obtained by the Naïve Bayes method was 96.08%. On the other hand, SVM and neural networks, the other two methods, produced an accuracy of 90.90%. False information has a big influence on a society's political atmosphere. Fake news has the ability to change people's opinions on social media. Without conducting any investigation, people change their minds in reaction to misleading information.

We need a technique that can recognize this kind of news. The researchers used machine learning classifiers for this. Many academics use the K-Nearest Neighbor, Support Vector Machine, Decision Tree, Linear Support Vector Machine, Logistic Regression, and Stochastic Gradient Descent classifiers. The results showed that the linear support vector machine was quite accurate at spotting false information. False news is recognized by machine learning classifiers. Different features have been used to train these classifiers. Training the classifiers is an important task since a trained classifier can yield more accurate results. False information may be detected more accurately by artificial intelligence. Fake news in Facebook postings has been detected using the Naïve Bayes classifier. This classifier gave them an accuracy rating of 74%. Artificial intelligence is more adept at identifying false information. The Naïve Bayes classifier has been employed to identify bogus news in Facebook posts. They received a 74% accuracy rate from this classifier, although they claimed that accuracy might be raised. These researchers also outline many approaches to increase accuracy in that study. Machine learning classifiers are available to identify false information.

Naive Bayes classifier

```
In [132]: from sklearn.naive_bayes import GaussianNB
          from sklearn.metrics import accuracy_score, confusion_matrix

          # Initialize Naive Bayes classifier
          nb_classifier = GaussianNB()

          # Fit the classifier to the training data
          nb_classifier.fit(X_train, y_train)
```

Out[132]: GaussianNB()
In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

Fig 30

Fig 30 shows us the Naive Bayes classifier used in our project.

```
In [133]: # Making predictions on the test data
          y_pred_nb = nb_classifier.predict(X_test)
```

```
In [134]: # Calculating accuracy
          accuracy_nb = accuracy_score(y_test, y_pred_nb)
          print("Accuracy:", accuracy_nb)
```

Accuracy: 0.9565817058369194

Fig 31

Fig 31 shows us the accuracy of the Naive Bayes classifier as 95%.

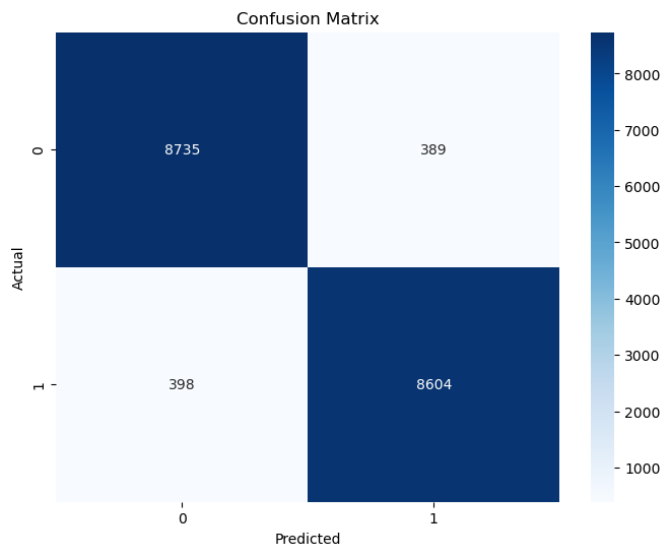


Fig 32

Fig 32 shows us the Naive Bayes confusion matrix built in our model.

```
In [136]: # Printing classification report
class_report = classification_report(y_test, y_pred_nb)
print("Classification Report:")
print(class_report)
```

```
Classification Report:
              precision    recall  f1-score   support

     0       0.96      0.96      0.96     9124
     1       0.96      0.96      0.96     9002

 accuracy          0.96
 macro avg          0.96
 weighted avg       0.96
```

Fig 33

Fig 33 shows us the classification report associated with the Naive Bayes model.

Decision Tree Classifier

One machine learning technique that is utilized for both regression and classification applications is the decision tree. It displays a structure like a flowchart, with each leaf node representing a choice based on its unique property and each node representing a conclusion. Based on the available informative attributes, the algorithm divides the dataset. The process of making decisions entails assessing each node's features and choosing the split that will maximize the information. The decision trees designate the majority of the class to each individual leaf in the classification tasks. The target variable is where the mean and median are predicted in regression problems. Decision trees are easily interpreted, shown, and represent the intricate relationships found in the data. The overfitting of the techniques available like pruning and overall decision tree provides an intuitive approach to it. Overall, the decision tree provides a proper approach to model the decision-making process and is widely used in the data analytics project.

Decision Tree classifier

```
In [137]: from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# Initialize Decision Tree classifier
dt_classifier = DecisionTreeClassifier()

# Fit the classifier to the training data
dt_classifier.fit(X_train, y_train)
```

Out[137]: DecisionTreeClassifier()

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

Fig 34

Fig 34 shows us the decision tree classifier used in our project.

```
In [138]: # Making predictions on the test data
y_pred_dt = dt_classifier.predict(X_test)
```

```
In [139]: # Calculate accuracy
accuracy_dt = accuracy_score(y_test, y_pred_dt)
print("Accuracy:", accuracy_dt)
```

Accuracy: 0.9132185810438045

Fig 35

Fig 35 shows us the accuracy of the decision tree classifier as 91%.

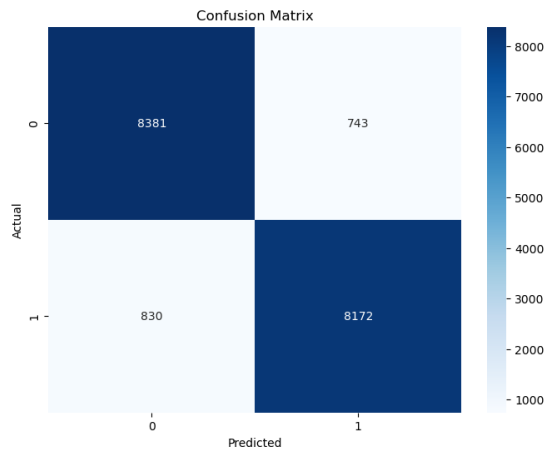


Fig 36

Fig 36 shows us the Decision tree confusion matrix built in our model.

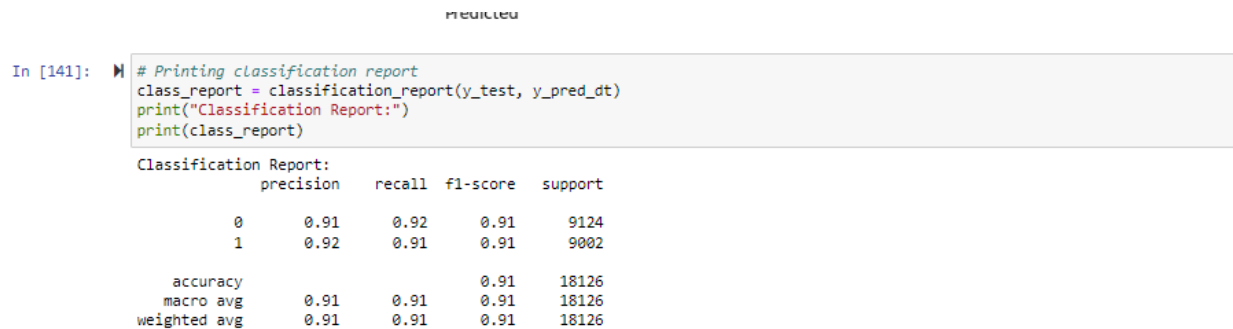


Fig 37

Fig 37 shows us the classification report associated with the Decision Tree.

From Figure 37, we can see that we got 91 percent accuracy for the decision tree classifier.

Logistic Regression

Logistic regression is the appropriate type of regression analysis to do when the dependent variable is binary. Like every other kind of regression analysis, logistic regression is one. It is used to describe data and make clear how one dependent binary variable and one or more independent variables at the nominal, ordinal, interval, or ratio level relate to each other.

Logistic Regression

```
In [142]: from sklearn.linear_model import LogisticRegression
          from sklearn.metrics import accuracy_score, confusion_matrix

          # Initialize Logistic Regression classifier
          lr_classifier = LogisticRegression()

          # Fit the classifier to the training data
          lr_classifier.fit(X_train, y_train)
```

C:\Users\RAJKUMAR\anaconda3\Lib\site-packages\sklearn\linear_model_logistic.py:460: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
n_iter_i = _check_optimize_result(

```
Out[142]: LogisticRegression()

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.
```

Fig 38

Fig 38 shows us the logistic regression used in our project.

```
In [143]: # Making predictions on the test data
          y_pred_lr = lr_classifier.predict(X_test)
```

```
In [144]: # Calculating accuracy
          accuracy_lr = accuracy_score(y_test, y_pred_lr)
          print("Accuracy:", accuracy_lr)
```

Accuracy: 0.9565817058369194

Fig 39

Fig 39 shows us the accuracy of logistics regression of 95%.

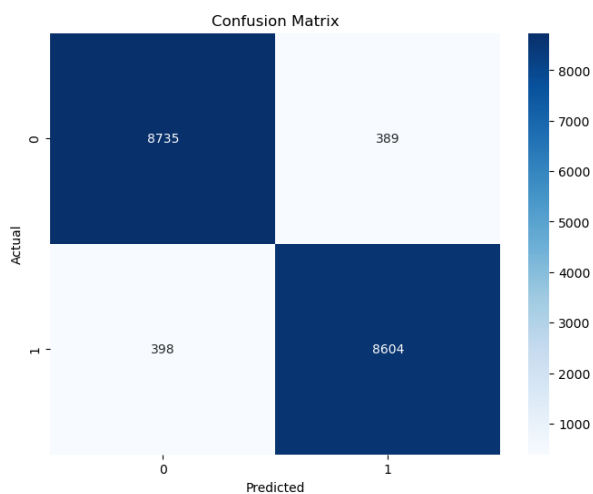


Fig 40

Fig 40 shows us the logistics regression confusion matrix.

```
In [146]: # Printing classification report
class_report = classification_report(y_test, y_pred_lr)
print("Classification Report:")
print(class_report)
```

Classification Report:				
	precision	recall	f1-score	support
0	0.96	0.96	0.96	9124
1	0.96	0.96	0.96	9002
accuracy			0.96	18126
macro avg	0.96	0.96	0.96	18126
weighted avg	0.96	0.96	0.96	18126

Fig 41

Fig 41 shows us the classification report in the logistics regression for 96%.

K-Nearest Neighbors (KNN) classifier

KNN is a straightforward machine learning technique that can be applied to both regression and classification problems. It works on the basis of the proximity principle, which states that the average of the nearby points or the majority of the class in the classification stage should be used to forecast a new data point. The closest neighbors taken into consideration for prediction-making are represented by the K in KNN. When used in a classification environment, KNN determines the class of a data point by comparing its k values to those of other classes specified by distance metrics like Euclidean distance.

Fake news identification is one of the most difficult tasks for humans. The use of machine learning simplifies the process of spotting fake news. There are several machine learning classifiers available to help assess the accuracy of news reports. These days, it's easy to collect the dataset required to train these classifiers. Many scholars used machine learning classifiers to confirm the accuracy of news reporting. the machine learning classifiers that detect false news. SVM and Naïve Bayes are the best classifiers for spotting fake news, according to the researchers' experiments. These two perform better than other classifiers based on the accuracy they provide. A classifier that is more accurate is thought to be better than others. The accuracy that each classifier provides is by far the most significant element. More erroneous information will be detected with the use of classifiers that are more accurate. Since so many individuals spread false information on social media to deceive others, it needs to be identified. Recognizing false news is crucial to shielding individuals or institutions from reputational damage. They contend that in this situation, machine learning is quite advantageous. After experimenting with a number of machine learning techniques, they found that logistic regression is the best classifier because of its higher accuracy. Social media posts are frequently made. Anyone may register on these websites and post any sort of stuff.

K-Nearest Neighbors (KNN) classifier

```
In [159]: from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# Initialize KNN classifier
knn_classifier = KNeighborsClassifier()

# Fit the classifier to the training data
knn_classifier.fit(X_train, y_train)
```

Out[159]: KNeighborsClassifier()

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

Fig 42

Fig 42 shows us the KNN regression used in our project.

```
In [160]: # Making predictions on the test data
y_pred_knn = knn_classifier.predict(X_test)
```

```
In [161]: # Calculating accuracy
accuracy_knn = accuracy_score(y_test, y_pred_knn)
print("Accuracy:", accuracy_knn)
```

Accuracy: 0.6525984773253889

Fig 43

Fig 43 shows us the accuracy of KNN regression of 65%

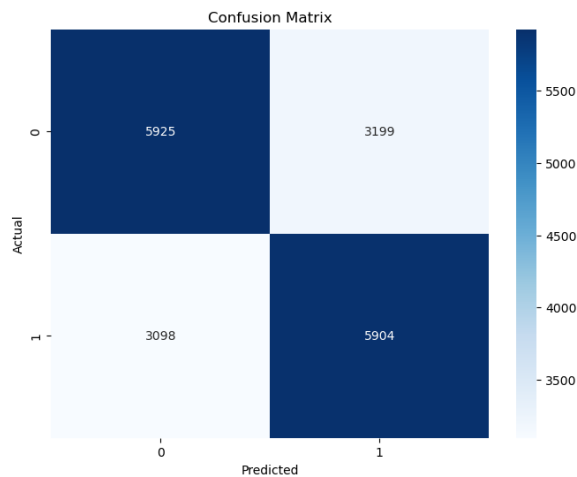


Fig 44

Fig 44 shows us the KNN regression confusion matrix.

```
In [163]: # Printing classification report
class_report = classification_report(y_test, y_pred_knn)
print("Classification Report:")
print(class_report)
```

Classification Report:				
	precision	recall	f1-score	support
0	0.66	0.65	0.65	9124
1	0.65	0.66	0.65	9002
accuracy			0.65	18126
macro avg	0.65	0.65	0.65	18126
weighted avg	0.65	0.65	0.65	18126

Fig 45

Fig 45 shows us the classification report of 65% in KNN regression.

Support Vector Machine (SVM) Classifier.

SVM is a powerful supervised technique that works better on smaller, more complex datasets. Support vector machines, or SVMs for short, are helpful in regression applications as well as classification applications, but their effectiveness is usually highest in the former. Finding the optimal hyperplane to split the two classes is the aim of this supervised machine learning problem. In SVM, we don't need to worry about extra observations since the margin is computed using the points that are closest to the hyperplane (support vectors); in logistic regression, on the other hand, the classifier is defined over all of the points. SVM hence operates at a naturally quicker speed.

Support Vector Machine (SVM) classifier

```
In [164]: # Initialize SVM classifier
svm_classifier = SVC()

# Fit the classifier to the training data
svm_classifier.fit(X_train, y_train)
```

Out[164]: SVC()
In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

Fig 46

Fig 46 shows us the SVM regression used in our project

```
In [165]: # Making predictions on the test data
y_pred_svm = svm_classifier.predict(X_test)
```

```
In [166]: # Calculating accuracy
accuracy_svm = accuracy_score(y_test, y_pred_svm)
print("Accuracy:", accuracy_svm)
```

Accuracy: 0.5488248924197285

Fig 47

Fig 47 shows us the accuracy of SVM regression of 54%.

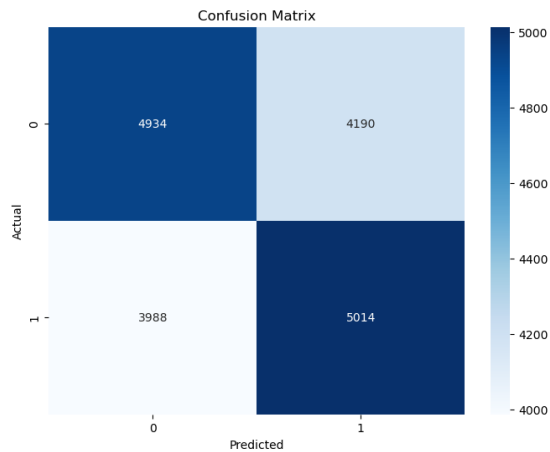


Fig 48

Fig 48 shows us the SVM regression confusion matrix.

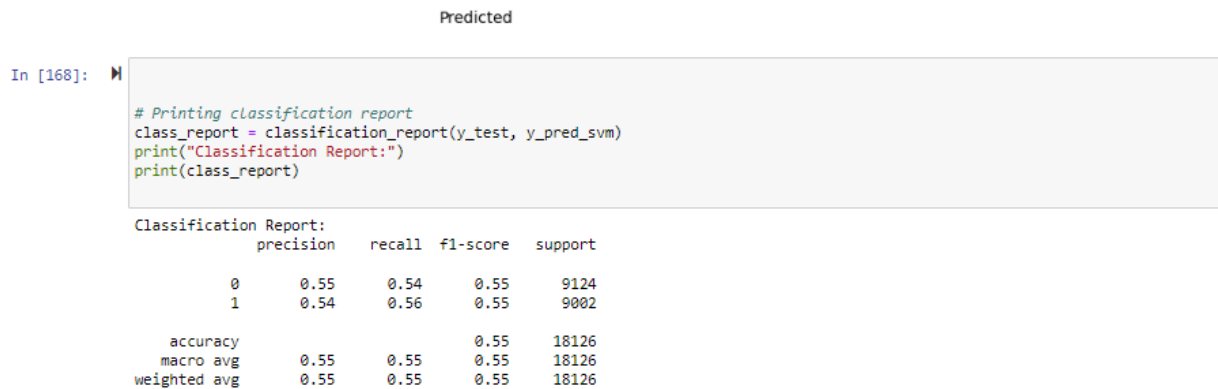


Fig 49

Fig 49 shows us the classification report of 55% in SVM regression.

Challenges Encountered

The process of cleaning and preprocessing the dataset came with several hurdles that demanded careful navigation to ensure all vital data was kept. Multiple features were removed from the multitude of features because they showed no relevance in detecting false news in tweets. Therefore, selecting the most relevant out of the several features proved to be a difficult task, in terms of aiming to strike a balance between sustaining data accessibility and eliminating noise and redundancy. Furthermore, the extraction and encoding of the textual data came with several

challenges on its own. Textual data often possess nuances and intricacy that require specialized methods to be able to pre-process the data set to a final analyzable form.

However, due to our mindful approach the datasets pivotal and insightful features remained in the dataset. Through careful removal of irrelevant, inconsistent, and redundant information the dataset had formed a strong foundation to begin accurately analyzing the dataset. Therefore, despite the array of challenges in preprocessing the data we were able to obtain vital observations into the prevalent signs that fake news tweets possessed. As a result, the cleaned and preprocessed data improved our research providing more efficient and less irrelevant features subsequently aiding in an accurate reliable machine learning model.

General Evaluation

The Internet is among the finest knowledge resources that people have access to. Many social media platforms, including Facebook and Twitter, let people connect with one other. The sharing of various news formats is also made easier by these platforms. People now days prefer to acquire their news from these platforms because of how accessible and easy to use they are. Another advantage for users of these platforms is their capacity to reply, comment, and do other things. These advantages are the reasons why people use these platforms. Cybercriminals, however, also exploit these networks as their finest source in addition to their advantages. Various persons have the ability to spread misleading information through various means. Various people have the ability to spread misleading information through various means. Additionally, these websites allow users to exchange news or articles, which is helpful for spreading misleading information.

People start to trust this kind of stuff, and they tell others as well. It's difficult to stop bogus news from spreading on these social media platforms. Anyone may sign up and start sharing news on these websites. It is possible to create a page that spreads misleading information and acts as a news source. These websites don't verify that the user is an authorized publisher. In this way, information against an individual or an organization may be distributed by everyone. These false stories may also have negative effects on society or a political party. Research indicates that spreading misleading information may quickly change people's opinions. Therefore, it is essential to halt the spread of these false allegations in order to protect the reputation of a person, political party, or organization.

False news may now be easily disseminated because of increased internet usage. Numerous social media platforms may be used to spread fake news to a wide audience. Thanks to the sharing features of these websites, information spread swiftly. In addition to an individual, a business or organization may also be impacted by false information. Limiting bogus news is therefore crucial. A person has to be well-informed on the topic in order to judge whether or not the news is untrue. It's a difficult task since most people don't know the complete story and jump to the conclusion that fake news is true without conducting any investigation. The issue of how to regulate fake news arises because it is uncontrollable. The answer lies in machine learning. Machine learning may be used to detect fake news. Machine learning may be used to swiftly and automatically identify these

fraudulent reports. After a post is made, machine learning algorithms will scan its content to find any instances of fake news. The best machine learning classifier for spotting fraudulent information is a topic of much research. It is important to consider the accuracy of the classifier since false information might negatively impact a large number of people if it is unable to distinguish it. The accuracy of the classifier is determined by its training procedure. An correctly trained model can yield higher results.

After intensive analysis including data preprocessing and exploratory data analysis (EDA), machine learning algorithms, and features correlation testing, our research enlightens the complex features of tweets makeup to predict the false information on social media platforms. Exploiting the diverse list of features, which include credibility scores, bot presence, and linguistic patterns, we uncover insightful characteristics associated with fake news tweets.

The accuracy of models being judged at the conclusion for the truth seeker dataset so we can conclude that the Random forest classifier, naive bayes classifier and logistic regression are having the same accuracy but after that the second place is decision tree classifier model with just a bit lacking, the third place of the accuracy goes to the KNN classifier model with up to 0.6 accuracy. The lowest of all the models used in our project and the least accuracy having is for the SVM classifier with 0.5 accuracy. Therefore, we can conclude that the random forest classifier, naive bayes classifier and logistic regression models will be best for analysis, in this project with 1 accuracy.

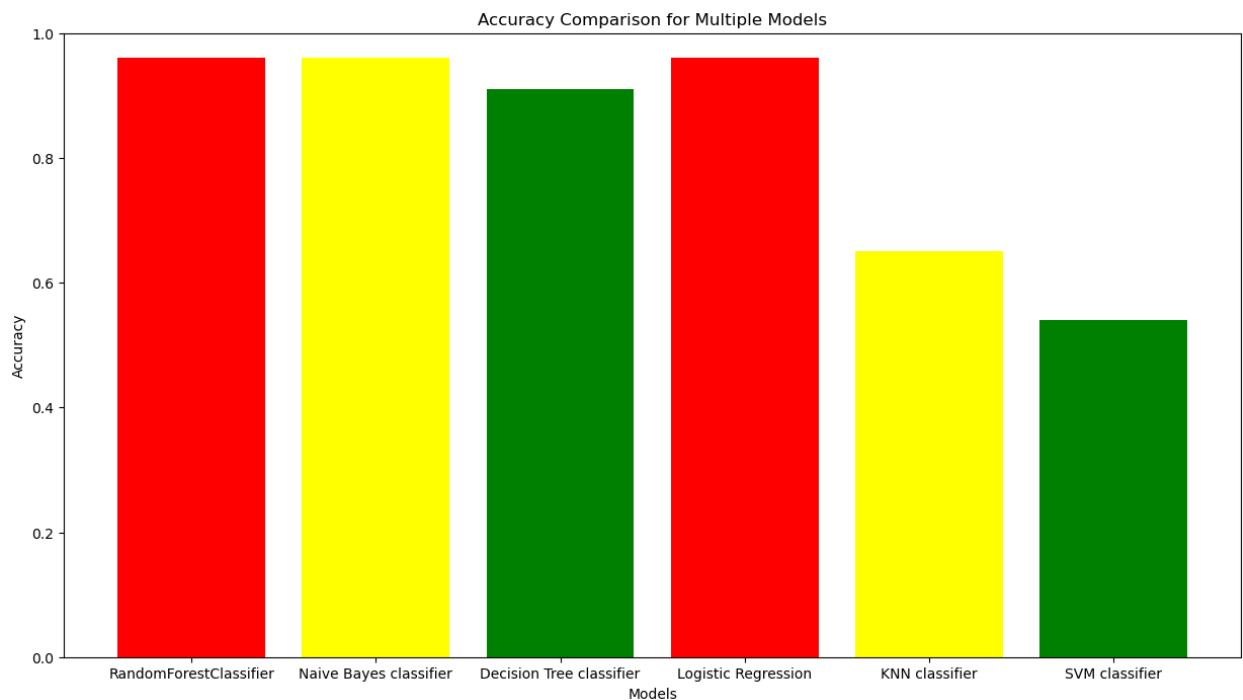


Fig 50

Fig 50 shows us the accuracy of the comparison models of different models used in our project.

References

- Saini, A. (2024, January 23). *Guide on Support Vector Machine (SVM) Algorithm*. Analytics Vidhya. <https://www.analyticsvidhya.com/blog/2021/10/support-vector-machinessvm-a-complete-guide-for-beginners/#:~:text=In%20machine%20learning%2C%20SVM%20is,complex%2C%20non%2Dlinear%20scenarios.>
- Saini, A. (2024b, April 4). *A Beginner's Guide to Logistic Regression*. Analytics Vidhya. <https://www.analyticsvidhya.com/blog/2021/08/conceptual-understanding-of-logistic-regression-for-data-science-beginners/>
- *Truth Seeker Dataset 2023 | Datasets | Research | Canadian Institute for Cybersecurity | UNB*. (n.d.-b). <https://www.unb.ca/cic/datasets/truthseeker-2023.html>
- https://www.researchgate.net/publication/370745691_TruthSeeker_The_Largest_Social_Media_Ground-Truth_Dataset_for_RealFake_Content
- Cococokoko. (n.d.). *GitHub - cococokoko/AWESOME-misinformation-detection: Detecting misleading tweets with Decision Tree, Random Forest, LassoCV, KNeighborsClassifier and OpenAI Api ada model*. GitHub. <https://github.com/cococokoko/AWESOME-misinformation-detection>
- *The largest social media Ground-Truth dataset for Real/Fake content: TruthSeeker*. (n.d.). IEEE Journals & Magazine | IEEE Xplore. <https://ieeexplore.ieee.org/abstract/document/10286332>
- https://easychair.org/publications/preprint_open/dCLN
- <https://www.techrxiv.org/users/691007/articles/681890-the-largest-social-media-ground-truth-dataset-for-real-fake-content-truthseeker>

- Abreu, D., Federal University of Pará (UFPA), Abelém, A., & Federal University of Pará (UFPA). (2023). *OMINACS: Online ML-Based IoT Network Attack Detection and Classification System* (p. 1) [Journal-article]. <https://arxiv.org/pdf/2302.09225.pdf>
- Pérez-Rosas, V., Kleinberg, B., Lefevre, A., & Mihalcea, R. (2018, August 1). *Automatic detection of fake news*. ACL Anthology. <https://aclanthology.org/C18-1287/>
- Singh, M. K., Ahmed, J., Alam, M. A., Raghuvanshi, K. K., & Kumar, S. (2023). A comprehensive review on automatic detection of fake news on social media. *Multimedia Tools and Applications*. <https://doi.org/10.1007/s11042-023-17377-4>
- Pérez-Rosas, V., Kleinberg, B., Lefevre, A., Mihalcea, R., Computer Science and Engineering, University of Michigan, & Department of Psychology, University of Amsterdam. (n.d.). Automatic detection of fake news. In *Computer Science and Engineering, University of Michigan* [Journal-article]. <https://web.eecs.umich.edu/~mihalcea/papers/perezrosas.coling18.pdf>
- Pérez-Rosas, V., Kleinberg, B., Lefevre, A., Mihalcea, R., Computer Science and Engineering, University of Michigan, & Department of Psychology, University of Amsterdam. (n.d.). Automatic detection of fake news. In *Computer Science and Engineering, University of Michigan* [Journal-article]. <https://web.eecs.umich.edu/~mihalcea/papers/perezrosas.coling18.pdf>
- Aldwairi, M., & Alwahedi, A. (2018). Detecting fake news in social media networks. *Procedia Computer Science*, 141, 215–222. <https://doi.org/10.1016/j.procs.2018.10.171>
- *Automatic detection of fake news on online platforms: A survey*. (2019, October 1). IEEE Conference Publication | IEEE Xplore. <https://ieeexplore.ieee.org/document/9002823>

Appendix

Python Code:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
pd.set_option('display.max_columns', None)
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
import string
import scipy.stats as stats

from sklearn.ensemble import RandomForestClassifier
import re
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.metrics import classification_report
from sklearn.svm import SVC

data = pd.read_csv("Features_For_Traditional_ML_Techniques.csv")
data.head()

# Displaying the shape of the DataFrame (number of rows and columns)
```

```

Data.shape

# Displaying information about the DataFrame, including column names, data types, and memory usage
data.info()

# Displaying the column names of the DataFrame
Data.columns

# Counting the number of null values in each column of the DataFrame
null_counts = data.isnull().sum()
print(null_counts)

# Displaying null values count for each column
for column, null_count in null_counts.items():
    print(f"{column}: {null_count}")

# Drop duplicates
data.drop_duplicates(inplace=True)
data.shape

# Selecting string columns from the DataFrame
string_columns = data.select_dtypes(include=['object'])

# Printing string columns
string_columns

# Selecting numerical columns from the DataFrame
numerical_columns = data.select_dtypes(include=['int64', 'float64'])

# Printing the numerical columns
numerical_columns

# Counting the number of string and numerical columns in the DataFrame
string_cols = 0
numerical_cols = 0

# Iterate through columns and check data types
for column in data.columns:
    if data[column].dtype == 'object':
        string_cols += 1
    else:
        numerical_cols += 1

# Print the counts
print("Number of string columns:", string_cols)
print("Number of numerical columns:", numerical_cols)

string_columns = data.select_dtypes(include=['object'])

# Iterating through string columns and print number of unique values
for column in string_columns.columns:
    unique_values = data[column].nunique() # Counting number of unique values in the column
    print(f"Column '{column}' has {unique_values} unique values.")

data["BinaryNumTarget"].value_counts()

```

```

data["majority_target"].value_counts()
data["followers_count"].value_counts()
data["following"].value_counts()

data.hist(figsize=(20,20), color='green')
plt.tight_layout()
plt.show

# Exclude boolean and object columns from histograms
numerical_columns = data.select_dtypes(include=['float64', 'int64']).columns

# Plotting histograms for numerical columns using a loop
for column in numerical_columns:
    plt.figure(figsize=(8, 6))
    data[column].hist(color='green')
    plt.title(f'Histogram of {column}')
    plt.xlabel(column)
    plt.ylabel('Frequency')
    plt.grid(False)
    plt.tight_layout()
    plt.show()

df = data.copy()
df.head(2)

# Exclude non-numeric columns and calculate correlation matrix
numeric_df = df.select_dtypes(include=['float64', 'int64'])
correlation_matrix = numeric_df.corr()
Correlation_matrix

# Identify Features with Low Correlation with Target Variable:
low_corr_features =
correlation_matrix['BinaryNumTarget'][correlation_matrix['BinaryNumTarget'].abs() < 0.1].index

# Drop Features with Low Correlation:
df.drop(low_corr_features, axis=1, inplace=True)
Df.columns
df = df.drop(["Unnamed: 0", "embeddings"], axis=1)
Df.columns
Df.shape
Data.columns
df['BotScoreBinary'] = data['BotScoreBinary']
df['followers'] = data['followers_count']
df['following'] = data['following']
df['past_tense_verbs'] = data['past_verbs']
df['present_tense_verbs'] = data['present_verbs']
Df.columns
Df.shape
df.head()

# Initialize WordNet Lemmatizer
lemmatizer = WordNetLemmatizer()
# Function to preprocess text
def preprocess_text(text):
    # Remove punctuation and convert to lowercase

```

```

    text = text.translate(str.maketrans('', '', string.punctuation)).lower()
    # Tokenization
    tokens = word_tokenize(text)
    # Remove stopwords
    stop_words = set(stopwords.words('english'))
    filtered_tokens = [word for word in tokens if word not in stop_words]
    # Lemmatization
    lemmatized_tokens = [lemmatizer.lemmatize(word) for word in filtered_tokens]
    # Join tokens back into a single string
    preprocessed_text = ' '.join(lemmatized_tokens)
    return preprocessed_text
Df[["statement", "tweet"]]

# Apply preprocessing to "statement" and "tweet" columns
import nltk
nltk.download('wordnet')
df['statement'] = df['statement'].apply(preprocess_text)
df['tweet'] = df['tweet'].apply(preprocess_text)
df.head()
Df[["statement", "tweet"]]
mapping = {True: 1, False: 0}
df["majority_target"] = df["majority_target"].map(mapping)
df['BinaryNumTarget'] = df['BinaryNumTarget'].astype(int)
df.head(2)
df.info()

from sklearn.feature_extraction.text import HashingVectorizer

# Initialize the HashingVectorizer
hashing_vectorizer = HashingVectorizer(n_features=1, norm=None, alternate_sign=False)

# Fit and transform the "statement" column
hashed_matrix = hashing_vectorizer.fit_transform(df["statement"])

# Convert the hashed matrix into a DataFrame
hashed_df = pd.DataFrame(hashed_matrix.toarray(), columns=["hashed_statement"])

# Concatenate the new hashed DataFrame with the original one
df = pd.concat([df, hashed_df], axis=1)

# Drop the original "statement" column if needed
df.drop("statement", axis=1, inplace=True)

from sklearn.feature_extraction.text import HashingVectorizer

# Initialize the HashingVectorizer
hashing_vectorizer = HashingVectorizer(n_features=1, norm=None, alternate_sign=False)

# Fit and transform the "statement" column
hashed_matrix = hashing_vectorizer.fit_transform(df["tweet"])

# Convert the hashed matrix into a DataFrame
hashed_df = pd.DataFrame(hashed_matrix.toarray(), columns=["hashed_tweet"])

# Concatenate the new hashed DataFrame with the original one

```

```

df = pd.concat([df, hashed_df], axis=1)

# Drop the original "statement" column if needed
df.drop("tweet", axis=1, inplace=True)

# Rename the column "hashed_statement" to "statement"
df.rename(columns={"hashed_statement": "statement"}, inplace=True)

# Rename the column "hashed_tweet" to "tweet"
df.rename(columns={"hashed_tweet": "tweet"}, inplace=True)
df.head()

# function to create histogram, Q-Q plot and
# boxplot.

def diagnostic_plots(df, variable):
    # function takes a dataframe (df) and
    # the variable of interest as arguments
    # define figure size
    plt.figure(figsize=(16, 4))

    # histogram
    plt.subplot(1, 3, 1)
    sns.histplot(df[variable], bins=30)
    plt.title('Histogram')

    # Q-Q plot
    plt.subplot(1, 3, 2)
    stats.probplot(df[variable], dist="norm", plot=plt)
    plt.ylabel('Variable quantiles')

    # boxplot
    plt.subplot(1, 3, 3)
    sns.boxplot(y=df[variable])
    plt.title('Boxplot')
    plt.show()

# let's find outliers in majority_target
diagnostic_plots(df, 'majority_target')
# visualise outliers in cred
diagnostic_plots(df, 'cred')
# visualise outliers in normalize_influence
diagnostic_plots(df, 'normalize_influence')

# visualise outliers in hashtags
diagnostic_plots(df, 'hashtags')

# visualise outliers in URLs
diagnostic_plots(df, 'URLs')

# visualise outliers in PERSON_percentage
diagnostic_plots(df, 'PERSON_percentage')

# visualise outliers in pronouns
diagnostic_plots(df, 'pronouns')

```

```

# visualise outliers in statement
diagnostic_plots(df, 'statement')

# visualise outliers in tweet
diagnostic_plots(df, 'tweet')

# visualise outliers in BotScoreBinary
diagnostic_plots(df, 'BotScoreBinary')

# visualise outliers in followers
diagnostic_plots(df, 'followers')

# visualise outliers in following
diagnostic_plots(df, 'following')
Df["present_tense_verbs"]

# visualise outliers in statement
diagnostic_plots(df, 'past_tense_verbs')

# visualise outliers in statement
diagnostic_plots(df, 'present_tense_verbs')

def find_skewed_boundaries(df, variable, distance):
    # Let's calculate the boundaries outside which sit the outliers
    # for skewed distributions
    # distance passed as an argument, gives us the option to
    # estimate 1.5 times the IQR to calculate
    # the boundaries.

    IQR = df[variable].quantile(0.75) - df[variable].quantile(0.25)
    lower_boundary = df[variable].quantile(0.25) - (IQR * distance)
    upper_boundary = df[variable].quantile(0.75) + (IQR * distance)
    return upper_boundary, lower_boundary
majority_target_upper_limit, majority_target_lower_limit = find_skewed_boundaries(df,
'majority_target', 1.5)
majority_target_upper_limit, majority_target_lower_limit

# find limits for cred
cred_upper_limit, cred_lower_limit = find_skewed_boundaries(df, 'cred', 1.5)
cred_upper_limit, cred_lower_limit

# find limits for normalize_influence
normalize_influence_upper_limit, normalize_influence_lower_limit = find_skewed_boundaries(df,
'normalize_influence', 1.5)
normalize_influence_upper_limit, normalize_influence_lower_limit

# find limits for hashtags
hashtags_upper_limit, hashtags_lower_limit = find_skewed_boundaries(df, 'hashtags', 1.5)
hashtags_upper_limit, hashtags_lower_limit

# find limits for URLs
URLs_upper_limit, URLs_lower_limit = find_skewed_boundaries(df, 'URLs', 1.5)
URLs_upper_limit, URLs_lower_limit

```

```

# find limits for PERSON_percentage
PERSON_percentage_upper_limit, PERSON_percentage_lower_limit = find_skewed_boundaries(df,
'PERSON_percentage', 1.5)
PERSON_percentage_upper_limit, PERSON_percentage_lower_limit

# find limits for pronouns
pronouns_upper_limit, pronouns_lower_limit = find_skewed_boundaries(df, 'pronouns', 1.5)
pronouns_upper_limit, pronouns_lower_limit

# find limits for statement
statement_upper_limit, statement_lower_limit = find_skewed_boundaries(df, 'statement', 1.5)
statement_upper_limit, statement_lower_limit

# find limits for tweet
tweet_upper_limit, tweet_lower_limit = find_skewed_boundaries(df, 'tweet', 1.5)
tweet_upper_limit, tweet_lower_limit

# find limits for BotScoreBinary
BotScoreBinary_upper_limit, BotScoreBinary_lower_limit = find_skewed_boundaries(df,
'BotScoreBinary', 1.5)
BotScoreBinary_upper_limit, BotScoreBinary_lower_limit

# find limits for followers
followers_upper_limit, followers_lower_limit = find_skewed_boundaries(df, 'followers', 1.5)
followers_upper_limit, followers_lower_limit

# find limits for past_tense_verbs
past_tense_verbs_upper_limit, past_tense_verbs_lower_limit = find_skewed_boundaries(df,
'past_tense_verbs', 1.5)
past_tense_verbs_upper_limit, past_tense_verbs_lower_limit

# find limits for present_tense_verbs
present_tense_verbs_upper_limit, present_tense_verbs_lower_limit = find_skewed_boundaries(df,
'present_tense_verbs', 1.5)
present_tense_verbs_upper_limit, present_tense_verbs_lower_limit

# find limits for following
following_upper_limit, following_lower_limit = find_skewed_boundaries(df, 'following', 1.5)
following_upper_limit, following_lower_limit

outliers_majority_target = np.where(df['majority_target'] > majority_target_upper_limit, True,
np.where(df['majority_target'] < majority_target_lower_limit, True, False))

outliers_cred = np.where(df['cred'] > cred_upper_limit, True,
np.where(df['cred'] < cred_lower_limit, True, False))
outliers_normalize_influence = np.where(df['normalize_influence'] >
normalize_influence_upper_limit, True,
np.where(df['normalize_influence'] < normalize_influence_lower_limit, True,
False))
outliers_hashtags = np.where(df['hashtags'] > hashtags_upper_limit, True,
np.where(df['hashtags'] < hashtags_lower_limit, True, False))

```

```

outliers_URLs = np.where(df['URLs'] > URLs_upper_limit, True,
                          np.where(df['URLs'] < URLs_lower_limit, True, False))
outliers_PERSON_percentage = np.where(df['PERSON_percentage'] > PERSON_percentage_upper_limit,
True,
                                     np.where(df['PERSON_percentage'] < PERSON_percentage_lower_limit, True,
False))
outliers_tweet = np.where(df['tweet'] > tweet_upper_limit, True,
                           np.where(df['tweet'] < tweet_lower_limit, True, False))
outliers_following = np.where(df['following'] > following_upper_limit, True,
                               np.where(df['following'] < following_lower_limit, True, False))
outliers_BotScoreBinary = np.where(df['BotScoreBinary'] > BotScoreBinary_upper_limit, True,
                                     np.where(df['BotScoreBinary'] < BotScoreBinary_lower_limit, True, False))
outliers_followers = np.where(df['followers'] > followers_upper_limit, True,
                               np.where(df['followers'] < followers_lower_limit, True, False))
outliers_past_tense_verbs = np.where(df['past_tense_verbs'] > past_tense_verbs_upper_limit, True,
                                      np.where(df['past_tense_verbs'] < past_tense_verbs_lower_limit, True,
False))
outliers_present_tense_verbs = np.where(df['present_tense_verbs'] >
present_tense_verbs_upper_limit, True,
                                       np.where(df['present_tense_verbs'] < present_tense_verbs_lower_limit, True,
False))
outliers_pronouns = np.where(df['pronouns'] > pronouns_upper_limit, True,
                              np.where(df['pronouns'] < pronouns_lower_limit, True, False))

outliers_statement = np.where(df['statement'] > statement_upper_limit, True,
                               np.where(df['statement'] < statement_lower_limit, True, False))
df_trimmed = df.loc[~(outliers_majority_target + outliers_cred +
outliers_normalize_influence+outliers_hashtags+outliers_URLs+outliers_PERSON_percentage+outliers_pronouns+outliers_statement+outliers_tweet+outliers_following+outliers_BotScoreBinary+outliers_followers+outliers_past_tense_verbs+outliers_present_tense_verbs), ]
df_trimmed.shape, df_trimmed.shape
df_trimmed.reset_index(drop=True, inplace=True)
df_trimmed.head()

Df_trimmed.shape

# Exclude non-numeric columns and calculate correlation matrix
numeric_df = df_trimmed.select_dtypes(include=['float64', 'int64'])
correlation_matrix = numeric_df.corr()
# Plot heatmap of correlation matrix
plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Heatmap')
plt.show()

Df_trimmed.columns

df_trimmed[['majority_target', 'BinaryNumTarget', 'cred',
            'normalize_influence', 'URLs', 'PERSON_percentage',
            'pronouns', 'followers', 'statement', 'tweet',
            'past_tense_verbs', 'present_tense_verbs']]

# Selecting the desired columns
selected_columns = ['majority_target', 'BinaryNumTarget', 'cred',

```

```

        'normalize_influence', 'URLs', 'PERSON_percentage',
        'pronouns', 'followers', 'statement', 'tweet',
        'past_tense_verbs', 'present_tense_verbs']

# Creating a correlation matrix for the selected columns
correlation_matrix = df_trimmed[selected_columns].corr()

# Plotting the heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
plt.title('Correlation Heatmap')
plt.show()

# Scatter plot between credibility scores and prevalence of fake news tweets
plt.figure(figsize=(8, 6))
plt.scatter(df_trimmed['cred'], df_trimmed['BinaryNumTarget'], color='green', alpha=0.5)
plt.title('Correlation between Credibility Scores and Prevalence of Fake News Tweets')
plt.xlabel('Credibility Scores')
plt.ylabel('Prevalence of Fake News Tweets')
plt.tight_layout()
plt.show()

# Separate features and target variable
X = df_trimmed.drop('BinaryNumTarget', axis=1)
y = df_trimmed['BinaryNumTarget']
# Initialize random forest classifier
rf_classifier = RandomForestClassifier()
# Fit the classifier to the data
rf_classifier.fit(X, y)
# Get feature importances
feature_importances = rf_classifier.feature_importances_
# Sort feature importances in descending order
sorted_indices = feature_importances.argsort()[::-1]
# Plot feature importances
plt.figure(figsize=(10, 8))
sns.barplot(x=feature_importances[sorted_indices], y=X.columns[sorted_indices], palette='coolwarm')
plt.title('Feature Importances for Predicting Fake News Tweets')
plt.xlabel('Importance Score')
plt.ylabel('Features')
plt.show()

# Calculating the probability of a fake news tweet being a bot
fake_news_bot_probability = df_trimmed[df_trimmed['BinaryNumTarget'] == 1]['BotScoreBinary'].mean()
# Printing the probability
print("Probability of a fake news tweet being a bot:", fake_news_bot_probability)
# Plot the relationship between BinaryNumTarget and BotScoreBinary
plt.figure(figsize=(10, 6))
sns.boxplot(x='BinaryNumTarget', y='BotScoreBinary', data=df_trimmed, palette='coolwarm')
plt.title('Relationship between Fake News Label and Bot Probability')
plt.xlabel('Fake News Label')
plt.ylabel('Bot Probability')
plt.xticks(ticks=[0, 1], labels=['Not Fake', 'Fake'])
plt.show()

# Calculate correlation matrix

```

```

correlation_matrix = df_trimmed.corr()
# Sort correlations with respect to BinaryNumTarget
correlation_with_target = correlation_matrix['BinaryNumTarget'].abs().sort_values(ascending=False)
# Display top correlated features
print(correlation_with_target)

# Scatter plot of followers vs following for fake news tweets
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x='followers', y='following', hue='BinaryNumTarget', palette='coolwarm')
plt.title('Relationship between Followers and Following for Fake News Tweets')
plt.xlabel('Number of Followers')
plt.ylabel('Number of Following')
plt.legend(title='Fake News')
plt.show()

# Grouped bar plot comparing percentage of past tense and present tense verbs for fake and non-fake
news tweets
plt.figure(figsize=(10, 6))

# Plotting past tense verbs
sns.barplot(x='BinaryNumTarget', y='past_tense_verbs', data=df_trimmed, color='blue', alpha=0.7,
label='Past Tense Verbs')
# Plotting present tense verbs
sns.barplot(x='BinaryNumTarget', y='present_tense_verbs', data=df_trimmed, color='red', alpha=0.7,
label='Present Tense Verbs')
plt.title('Comparison of Percentage of Past Tense and Present Tense Verbs for Fake and Non-Fake
News Tweets')
plt.xlabel('Fake News')
plt.ylabel('Percentage of Verbs')
plt.xticks([0, 1], ['Non-Fake', 'Fake'])
plt.legend()
plt.show()

Df_trimmed.columns
# Separating features (X) and target variable (y)
X = df_trimmed.drop('BinaryNumTarget', axis=1)
y = df_trimmed['BinaryNumTarget']

# Split the data into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialize Random Forest classifier
rf_classifier = RandomForestClassifier(random_state=42)
# Fit the classifier to the training data
rf_classifier.fit(X_train, y_train)

# Predicting the target variable on the test set
y_pred = rf_classifier.predict(X_test)

# Calculating accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Printing confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)

```

```

print("Confusion Matrix:")
print(conf_matrix)

# Plotting confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()

# Printing classification report
class_report = classification_report(y_test, y_pred)
print("Classification Report:")
print(class_report)

from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix

# Initialize Naive Bayes classifier
nb_classifier = GaussianNB()

# Fit the classifier to the training data
nb_classifier.fit(X_train, y_train)

# Making predictions on the test data
y_pred_nb = nb_classifier.predict(X_test)

# Calculating accuracy
accuracy_nb = accuracy_score(y_test, y_pred_nb)
print("Accuracy:", accuracy_nb)

# Printing confusion matrix
conf_matrix_nb = confusion_matrix(y_test, y_pred_nb)

# Plot confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_nb, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()

# Printing classification report
class_report = classification_report(y_test, y_pred_nb)
print("Classification Report:")
print(class_report)

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
# Initialize Decision Tree classifier
dt_classifier = DecisionTreeClassifier()
# Fit the classifier to the training data
dt_classifier.fit(X_train, y_train)

```

```

# Making predictions on the test data
y_pred_dt = dt_classifier.predict(X_test)

# Calculate accuracy
accuracy_dt = accuracy_score(y_test, y_pred_dt)
print("Accuracy:", accuracy_dt)

# Printing confusion matrix
conf_matrix_dt = confusion_matrix(y_test, y_pred_dt)
# Plot confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_dt, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()

# Printing classification report
class_report = classification_report(y_test, y_pred_dt)
print("Classification Report:")
print(class_report)

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
# Initialize Logistic Regression classifier
lr_classifier = LogisticRegression()
# Fit the classifier to the training data
lr_classifier.fit(X_train, y_train)

# Making predictions on the test data
y_pred_lr = lr_classifier.predict(X_test)

# Calculating accuracy
accuracy_lr = accuracy_score(y_test, y_pred_lr)
print("Accuracy:", accuracy_lr)

# Printing confusion matrix
conf_matrix_lr = confusion_matrix(y_test, y_pred_lr)
# Plot confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_lr, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()

# Printing classification report
class_report = classification_report(y_test, y_pred_lr)
print("Classification Report:")
print(class_report)

from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
# Initialize KNN classifier
knn_classifier = KNeighborsClassifier()

```

```

# Fit the classifier to the training data
knn_classifier.fit(X_train, y_train)

# Making predictions on the test data
y_pred_knn = knn_classifier.predict(X_test)

# Calculating accuracy
accuracy_knn = accuracy_score(y_test, y_pred_knn)
print("Accuracy:", accuracy_knn)

# confusion matrix
conf_matrix_knn = confusion_matrix(y_test, y_pred_knn)
# Plot confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_knn, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()

# Printing classification report
class_report = classification_report(y_test, y_pred_knn)
print("Classification Report:")
print(class_report)

# Initialize SVM classifier
svm_classifier = SVC()
# Fit the classifier to the training data
svm_classifier.fit(X_train, y_train)

# Making predictions on the test data
y_pred_svm = svm_classifier.predict(X_test)

# Calculating accuracy
accuracy_svm = accuracy_score(y_test, y_pred_svm)
print("Accuracy:", accuracy_svm)

# Printing confusion matrix
conf_matrix_svm = confusion_matrix(y_test, y_pred_svm)
# Plot confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_svm, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()

# Printing classification report
class_report = classification_report(y_test, y_pred_svm)
print("Classification Report:")
print(class_report)

import matplotlib.pyplot as plt

# Example: Multiple accuracy values for different models or parameters

```

```
model_names = ['RandomForestClassifier','Naive Bayes classifier','Decision Tree
classifier','Logistic Regression','KNN classifier','SVM classifier']
accuracy_values = [0.96, 0.96, 0.91, 0.96,0.65,0.54]

# Plotting the accuracy values
plt.figure(figsize=(15, 8))
plt.bar(model_names, accuracy_values, color=['red', 'yellow', 'green'])
plt.xlabel('Models')
plt.ylabel('Accuracy')
plt.title('Accuracy Comparison for Multiple Models')
plt.ylim(0, 1) # Set the y-axis limit to represent accuracy values between 0 and 1
plt.show()
```